

Temario completo - Curso de Analista de Datos con Python

Bloque 00 - Orientación y pensamiento analítico

Propósito

Este bloque enseña a pensar antes de abrir una hoja de cálculo, Python o una herramienta de BI. El resultado esperado es poder convertir una petición vaga en una pregunta analítica que otra persona pueda revisar y usar para decidir.

Lecciones

1. [El rol del analista y el ciclo de decisión](#)
2. [Preguntas, hipótesis, evidencia y métricas](#)
3. [Cómo estudiar y trabajar de forma reproducible](#)

Prerrequisitos

Ninguno. No hace falta saber programación ni conocer vocabulario técnico.

Ejercicios

Realiza [los ejercicios de comprensión](#) antes de consultar [las soluciones](#).

El rol del analista y el ciclo de decisión

Objetivos y punto de partida

Al terminar podrás distinguir una decisión de un cálculo y describir el trabajo de un analista sin reducirlo a "hacer gráficos". No necesitas saber qué es una base de datos ni programar.

El problema que resuelve el análisis

Una empresa de reparto observa menos pedidos que el mes anterior. La directora debe decidir si cambia una campaña, corrige una incidencia en la aplicación o no hace nada porque la variación es normal. Mirar un número aislado no responde esa decisión: hace falta saber **qué cambió, para quién, respecto a qué referencia y con qué confianza**.

Un **analista de datos** transforma una necesidad de decisión en evidencia revisable. Su producto final no es una tabla ni un gráfico: es una recomendación que explica qué se sabe, qué no se sabe y qué conviene comprobar después.

Del encargo a la acción

Antes de hablar de herramientas, mira este recorrido. Responde a la pregunta: “¿cómo se evita saltar de una impresión a una conclusión?”



El ciclo es deliberadamente circular. Una recomendación no cierra el trabajo: genera una acción cuya consecuencia debe medirse. Si la campaña cambia, habrá que volver a comprobar pedidos, margen y posibles efectos no deseados.

Ejemplo trabajado

Petición inicial: “las ventas van mal”.

Una respuesta precipitada sería abrir un gráfico mensual y proponer bajar precios. Una respuesta analítica empieza preguntando: “¿qué decisión está sobre la mesa?”. Supongamos que la decisión es redistribuir 20 000 euros de publicidad. Entonces la pregunta puede ser: “¿Qué canal explica la caída de pedidos de junio frente al promedio de marzo a mayo, y cuál mantiene margen suficiente para recibir inversión?”

La pregunta obliga a definir después “pedido”, “canal”, “margen” y los periodos. Aún no demuestra que un canal **cause** la caída; solo delimita qué evidencia buscar.

Límite importante: describir no es explicar causas

Que los pedidos hayan bajado después de una campaña no prueba que la campaña sea la causa. Puede haber estacionalidad, un fallo técnico, cambios de precio o clientes distintos. El análisis

descriptivo dice qué ocurrió; una explicación causal exige comparaciones y diseño más cuidadoso, que se estudiará más adelante.

Resumen y comprobación

- El analista ayuda a decidir con evidencia trazable.
- Una cifra es un indicio, no una conclusión.
- El trabajo continúa después de recomendar: hay que medir el resultado.

Pregúntate: ¿qué decisión cambiaría si el análisis mostrara que la caída procede solo de usuarios nuevos? ¿Qué dato adicional pedirías antes de recomendar bajar precios?

Sigue con [preguntas](#), [hipótesis](#), [evidencia y métricas](#) y después resuelve el [ejercicio del bloque](#).

Preguntas, hipótesis, evidencia y métricas

Objetivos y prerequisites

Aprenderás a convertir una petición ambigua en una pregunta medible, a separar una hipótesis de un hecho y a pedir una métrica bien definida. Parte de la lección anterior; no requiere matemáticas avanzadas.

Una pregunta que se pueda comprobar

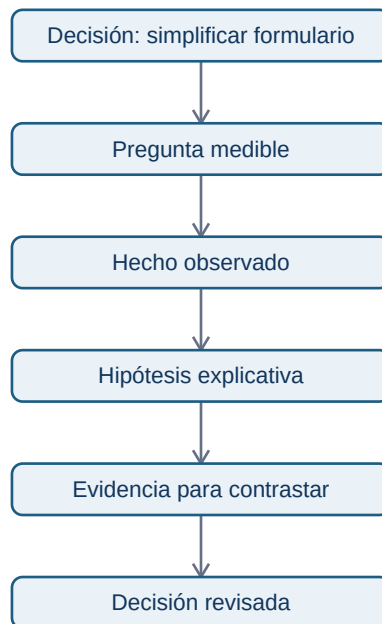
“Queremos que más personas usen la aplicación” expresa un deseo, no una pregunta. Para que un equipo pueda investigar necesita concretar cuatro piezas: la **población** (quiénes), el **resultado** que se observa, el **periodo** y la **comparación**.

Ejemplo: “Entre quienes instalaron la aplicación en mayo, ¿qué porcentaje completó su primera reserva en siete días, frente a quienes la instalaron en abril?”. Ya se puede buscar evidencia, y también se puede discutir si siete días es una ventana razonable.

Hecho, hipótesis y evidencia

Una **hipótesis** es una explicación provisional que podría ser falsa: “el formulario largo reduce las reservas”. La **evidencia** es la información que apoya o cuestiona esa explicación: registros de pasos completados, una comparación entre versiones o entrevistas. Un resultado observado es: “solo el 42 % llega al último paso”. No confundas el resultado con su causa.

Este mapa responde a “¿qué hay que mantener separado para razonar con rigor?”



La flecha no convierte una hipótesis en verdad. La evidencia puede hacerla más plausible, descartarla o mostrar que faltan datos.

Qué significa “métrica”

Una **métrica** es una medida cuya regla se puede repetir. “Usuarios activos” no basta si nadie sabe si cuenta una apertura de la app, una reserva, un día o un mes. Una definición mínima incluye fórmula, población, periodo, fuente y responsable.

Por ejemplo: “tasa de primera reserva a siete días = personas que reservan en los siete días posteriores a instalar / personas que instalan, para instalaciones de mayo, excluyendo cuentas de prueba”. Otra persona debe llegar al mismo número al aplicar la misma regla.

Un **KPI** es una métrica elegida para seguir un objetivo importante. No todo número es KPI: si una métrica no orienta una decisión, quizá es ruido o contexto, no una prioridad.

Error habitual: optimizar el número equivocado

Si se premia solo “reservas creadas”, un equipo podría facilitar reservas que luego se cancelan. Por eso una métrica principal suele necesitar una métrica de protección: junto a reservas, vigilar cancelaciones, reclamaciones o margen. Este riesgo se retomará al diseñar árboles de métricas.

Resumen y práctica

- Una pregunta comprobable delimita población, resultado, periodo y comparación.
- Una hipótesis explica provisionalmente; la evidencia la contrasta.
- Una métrica reproducible tiene una definición, no solo un nombre.

Redacta una pregunta sobre el uso de una app y añade una hipótesis alternativa que también pudiera explicar el resultado. Después completa [los ejercicios](#).

Cómo estudiar y trabajar de forma reproducible

Objetivos y prerequisites

Sabrás organizar una sesión de estudio desde el móvil o un ordenador, usar ayuda de AI sin delegar el razonamiento y dejar rastro de tus decisiones. No hace falta instalar nada.

Teoría, práctica y recuperación

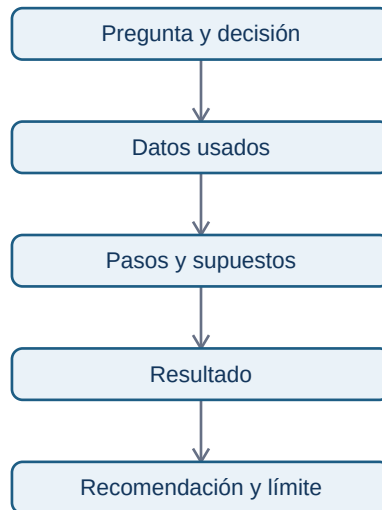
Leer una explicación da familiaridad, pero no garantiza que puedas aplicarla. Alterna tres acciones: comprender un ejemplo, resolver uno parecido sin mirar y explicar con tus palabras por qué la respuesta tiene sentido. Ese último paso descubre lagunas antes de que se conviertan en hábitos.

Una sesión mínima desde el móvil puede consistir en leer una lección, responder dos preguntas de comprobación en notas y revisar la solución solo después. Cuando tengas navegador con teclado, abre el notebook de práctica en Google Colab: un **notebook** es una página que mezcla texto, código ejecutable y resultados; lo estudiarás desde cero en Python.

Reproducibilidad: poder revisar el camino

Un análisis es **reproducible** si otra persona puede entender qué datos se usaron, qué reglas se aplicaron y cómo se llegó a una conclusión. No requiere una herramienta sofisticada al principio: basta con anotar la pregunta, la fecha, los supuestos y los cambios.

El siguiente flujo responde a “¿qué debe conservarse para que una conclusión sea revisable?”



Si falta un eslabón, alguien puede ver un número final pero no comprobar si era apropiado. Más adelante Git, tickets de Jira y documentación permitirán hacer este proceso colaborativo.

Usar AI como tutor, no como piloto automático

Una buena petición a una AI incluye contexto: “No sé qué es una lista en Python; explícame este error con un ejemplo de gastos y pregúntame después”. Pide que explique cada línea y cambia un valor para comprobar que entiendes el efecto. Nunca des por correcta una consulta, gráfico o conclusión solo porque suena convincente: ejecútala, verifica los supuestos y conserva la fuente.

Diagnóstico inicial y ruta adaptable

Si ya manejas porcentajes, medias o álgebra, puedes leer el bloque de matemáticas como repaso y dedicar más atención a sus aplicaciones analíticas. No conviene saltarse la definición de métricas, incertidumbre o sesgo: conocer la fórmula no garantiza interpretar bien un dato empresarial.

Resumen y siguiente paso

- Estudiar implica recuperar y aplicar, no solo leer.
- Reproducibilidad conserva pregunta, datos, pasos, resultado y límites.
- AI acelera la práctica si puedes explicar y verificar su respuesta.

Completa el [diagnóstico inicial](#). Después continúa con el [bloque 01](#): allí se explicará primero qué es un archivo, una tabla y una observación.

Bloque 01 - Fundamentos de datos: desde archivos hasta calidad

Propósito

Antes de programar o abrir una base de datos, Leo debe entender qué representa un dato, cómo se organiza la información y por qué una cifra aparentemente correcta puede conducir a una decisión equivocada. Este bloque no presupone que sepas qué es CSV, JSON, una tabla o una clave.

Resultado de salida

Al terminar podrás abrir un archivo sencillo y explicar qué información contiene, distinguir una tabla de un documento JSON, identificar el grano de un conjunto de datos, proponer controles de calidad y reconocer límites de privacidad y sesgo.

Lecciones

1. Qué es un dato, un archivo y una tabla.
2. Filas, columnas, tipos y relaciones.
3. CSV, JSON, Excel, Parquet y bases de datos.
4. Calidad, ausencia, sesgo, privacidad y ética.

Práctica

Realiza [la auditoría de calidad](#) después de la lección 4 y compárala con [la solución razonada](#).

01.1 Qué es un dato, un archivo y una tabla

Objetivos

Entender qué es un dato antes de hablar de herramientas; distinguir una información suelta de un conjunto organizado; y reconocer qué representa una fila de una tabla.

Empieza por una situación cotidiana

Imagina una tienda que quiere saber qué productos se venden más. Cada vez que una persona compra algo, la tienda puede guardar información: fecha, producto, importe y forma de pago. Cada una de esas piezas es un **dato**: una representación de algo que ocurrió en el mundo real.

La información se guarda en un **archivo**, igual que una fotografía o una nota de texto. Un archivo tiene nombre, contenido y un formato que indica cómo se organiza. No hay magia: un archivo de datos es una manera de guardar información para poder leerla, compartirla o analizarla después.

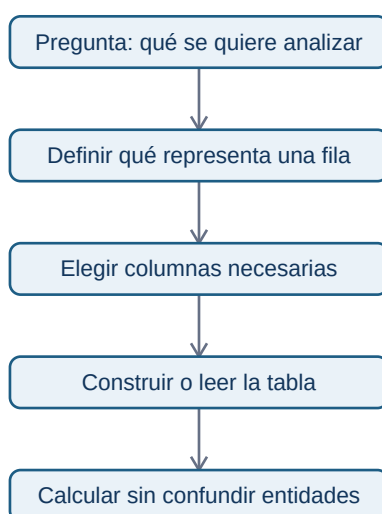
Una de las formas más comunes de organizar datos es una **tabla**. Una tabla se parece a una hoja de cálculo: tiene columnas que describen qué tipo de información guardamos y filas que recogen casos concretos.

fecha	producto	importe	canal
2026-01-03	teclado	45.99	web
2026-01-04	ratón	19.90	tienda
2026-01-04	teclado	45.99	web

En este ejemplo, la primera fila de datos representa una compra concreta. La columna **producto** responde qué se compró; **importe**, cuánto costó. La tabla no “sabe” qué significa un teclado: nosotros le damos significado a las columnas.

El grano: la pregunta que evita errores grandes

El **grano** indica qué representa exactamente una fila. Aquí, una fila representa una compra, no un cliente ni un producto. Esta frase parece pequeña, pero evita errores muy caros: si se cuenta una fila como si fuera una persona, un cliente que compra tres veces se contará como tres clientes.



Antes de cualquier análisis, completa siempre esta frase: “cada fila de este conjunto representa...”. Si no puedes completarla, no empieces a sumar ni a calcular promedios.

Ejemplo IT

Una aplicación registra eventos. Una tabla puede tener una fila por clic, otra por sesión y otra por usuario. Son tablas distintas con granos distintos. Contar clics no responde cuántos usuarios usaron la aplicación; contar sesiones tampoco responde directamente cuántas personas pagaron.

Error frecuente

Pensar que una tabla es “la realidad”. Una tabla es un modelo parcial. Puede no incluir compras hechas por teléfono, usuarios anónimos o devoluciones. Siempre pregunta qué no está en los datos.

Comprobación

Para una academia online, escribe el grano de tres tablas posibles: una de alumnos, una de inscripciones y una de visualizaciones de vídeo. ¿Por qué no deben mezclarse sin una relación explícita?

01.2 Filas, columnas, tipos y relaciones

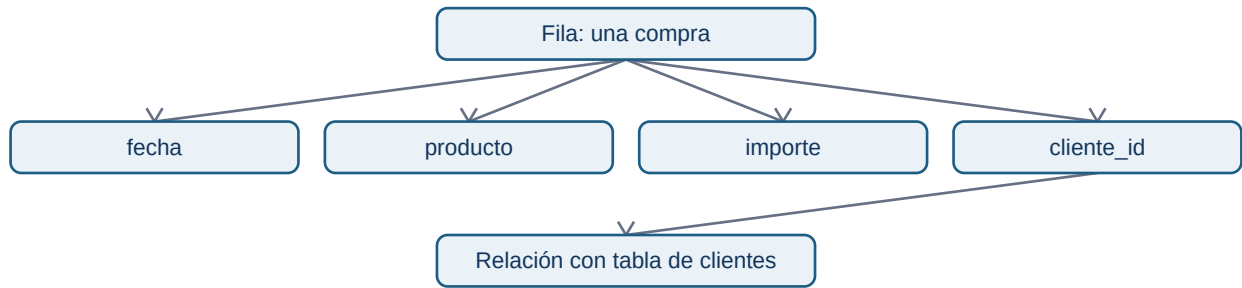
Objetivos

Aprender a leer una tabla con precisión: distinguir filas y columnas, reconocer tipos básicos de datos y entender por qué una clave permite relacionar tablas sin duplicar significado.

Filas y columnas no son solo una forma visual

Una **fila** contiene información de un caso. Una **columna** guarda el mismo tipo de atributo para muchos casos. En una tabla de compras, `importe` debería contener números; en una tabla de usuarios, `fecha_registro` debería contener fechas. El tipo de información determina qué operaciones tienen sentido.

No tiene sentido calcular la media de ciudad; sí puede tener sentido contar cuántos usuarios hay por ciudad. No tiene sentido ordenar alfabéticamente un importe para encontrar la venta mayor; sí tiene sentido convertirlo a número y compararlo.



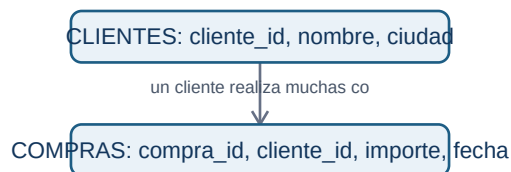
Tipos que encontrarás al empezar

- **Texto:** nombre de producto, ciudad, comentario.
- **Número:** importe, cantidad, edad, latencia.
- **Fecha y hora:** momento de registro, compra o despliegue.
- **Booleano:** verdadero/falso; por ejemplo, es_cliente.
- **Categoría:** conjunto limitado de etiquetas, como plan gratis, pro o empresa.

Un número puede representar cosas distintas: un identificador `cliente_id=1042` parece número, pero no debes calcular su media. Es una etiqueta técnica, no una cantidad.

Claves y relaciones

Una **clave** es una columna que permite identificar o conectar información. Si `cliente_id` identifica de forma única a cada cliente, se llama clave primaria de la tabla de clientes. La misma columna puede aparecer en la tabla de compras para indicar quién realizó cada compra; allí actúa como clave foránea o referencia.



La relación dice: un cliente puede realizar muchas compras; una compra pertenece a un cliente. Esta información es esencial al combinar tablas. Si una tabla de clientes tiene accidentalmente dos filas para el mismo `cliente_id`, una unión puede duplicar importes sin que el error sea evidente.

Error frecuente

Confundir un identificador con una medida. `pedido_id` y `cliente_id` sirven para identificar; no para hacer promedios. También es un error asumir que una columna “única” realmente lo es sin comprobar duplicados y nulos.

Comprobación

Diseña las columnas mínimas de una tabla de tickets de soporte. ¿Qué representa cada fila? ¿Qué columna conectaría un ticket con un cliente? ¿Cuál parece numérica pero no debe tratarse como medida?

01.3 CSV, JSON, Excel, Parquet y bases de datos

Objetivos

Saber qué problema resuelve cada formato básico y elegir una forma razonable de guardar o recibir información sin memorizar una lista de siglas.

CSV: una tabla escrita como texto

Un archivo **CSV** significa **Comma-Separated Values**: valores separados por comas. Es un archivo de texto que representa una tabla. La primera línea suele contener los nombres de las columnas; cada línea posterior es una fila.

```
fecha, producto, importe
2026-01-03, teclado, 45.99
2026-01-04, ratón, 19.90
```

CSV es sencillo de abrir, compartir y leer con Python, Excel o un editor de texto. Su simplicidad también tiene límites: no guarda bien tipos complejos, fórmulas, varias hojas ni una estructura dentro de otra. Además, hay que acordar separador, codificación, formato de fecha y separador decimal.

JSON: una ficha que puede contener otras fichas

JSON significa **JavaScript Object Notation**. Es texto estructurado mediante pares `campo: valor`. A diferencia de un CSV, puede guardar objetos dentro de objetos y listas. Es frecuente en APIs, configuraciones y eventos de aplicaciones.

```
{
  "pedido_id": 1001,
```

```

"cliente": {
  "nombre": "Leo",
  "ciudad": "Madrid"
},
"productos": ["teclado", "ratón"],
"importe": 65.89
}

```

El JSON anterior es una ficha de pedido. No es una tabla, aunque se pueda transformar en una. Para analizar muchos pedidos, normalmente tendrás que decidir qué campos extraer y cómo convertir listas u objetos anidados en columnas o tablas relacionadas.



Excel, Parquet y bases de datos

Excel es una aplicación y un formato de libro de trabajo; es útil para revisión manual, cálculos ligeros y comunicación. No es ideal como fuente única de procesos repetibles si múltiples personas lo editan sin control.

Parquet es un formato optimizado para datos tabulares grandes. Guarda columnas de forma eficiente y conserva tipos mejor que CSV. Normalmente lo usarás mediante Python, Spark, DuckDB o un warehouse, no editándolo a mano.

Una **base de datos** organiza información para que aplicaciones y personas puedan consultarla con reglas de acceso, relaciones y actualizaciones. SQL es un lenguaje para consultar muchas bases relacionales; MongoDB almacena documentos similares a JSON; DynamoDB se diseña alrededor de claves y patrones de acceso.

Cómo elegir sin obsesionarte

Empieza preguntando: ¿necesito una tabla simple que cualquiera pueda abrir? CSV. ¿Recibo una respuesta con estructuras anidadas de una API? JSON. ¿Trabajo con muchos datos tabulares repetidamente? Parquet o una base de datos. ¿Necesito revisar manualmente algo pequeño? Excel puede ser adecuado.

La elección no elimina el deber de conocer grano, calidad y significado.

Comprobación

Explica a otra persona la diferencia entre CSV y JSON sin usar las palabras “plano” ni “anidado”. Después indica cuál esperarías recibir de una API meteorológica y por qué.

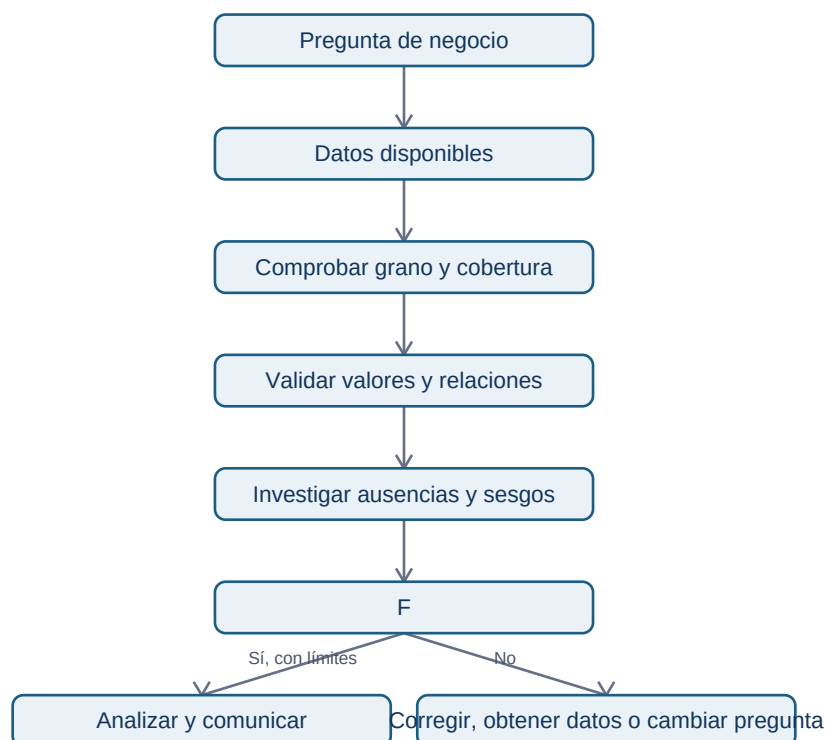
01.4 Calidad, ausencia, sesgo, privacidad y uso responsable

Objetivos

Revisar un conjunto de datos antes de analizarlo, interpretar ausencias sin borrarlas por costumbre y reconocer que una decisión basada en datos también puede causar daño.

Calidad como condición de confianza

La calidad no significa que un dataset sea perfecto. Significa que conocemos si es adecuado para una decisión concreta. Una tabla de compras puede ser suficiente para estimar ingresos diarios y no serlo para saber satisfacción de clientes.



Cinco controles iniciales son especialmente útiles:

- **Compleitud:** ¿faltan valores necesarios para la pregunta?
- **Validez:** ¿los valores respetan reglas, unidades y formatos?
- **Consistencia:** ¿la misma idea está registrada de la misma forma?
- **Unicidad:** ¿existen duplicados indebidos?
- **Actualidad:** ¿el dato llega a tiempo para la decisión?

Los nulos cuentan una historia

Un valor ausente no es automáticamente un error. Puede significar “no se aplica”, “no se midió”, “falló el sistema” o “la persona prefirió no responder”. Borrar todas las filas con nulos puede eliminar justo a la población con la que tienes un problema.

Por ejemplo, si el campo ingresos falta sobre todo en usuarios que abandonan un formulario, la ausencia es información sobre fricción. Antes de imputar o eliminar, mide dónde faltan datos, desde cuándo y en qué segmentos.

Sesgo, privacidad y propósito

Un dataset puede representar peor a grupos que usan menos una aplicación, tienen conectividad limitada o no están incluidos en la fuente. Un modelo entrenado con esos datos puede amplificar esa desigualdad. El analista debe declarar cobertura, exclusiones y riesgos, no tratarlos como una nota al pie.

La privacidad comienza antes de abrir un archivo: recoge solo los datos necesarios, evita copiar identificadores personales en notebooks, limita acceso y define cuánto tiempo se conservan. Que un sistema permita acceder a una columna no significa que sea legítimo usarla para cualquier objetivo.

Caso práctico

Una empresa quiere comparar uso por ciudad, pero el 30 % de usuarios no informa ciudad y ese porcentaje es mayor en móvil. Concluir que “móvil usa menos el producto en ciertas ciudades” sin estudiar la ausencia puede ser falso. Primero se investiga el formulario, la geolocalización, el consentimiento y los segmentos afectados.

Comprobación

Elige una de las cinco dimensiones de calidad y describe: un error concreto, cómo lo detectarías, qué decisión podría dañar y cuál sería una respuesta prudente.

Bloque 02 - Python desde cero

Propósito

Aprender a leer, escribir y depurar programas pequeños antes de usar bibliotecas de análisis. El objetivo no es memorizar sintaxis: es expresar una regla de negocio de forma clara y verificable.

Lecciones

1. Ejecutar código, valores y variables
2. Listas, diccionarios y datos sencillos
3. Decisiones y repeticiones
4. Funciones y alcance
5. Errores y depuración
6. Estilo y práctica guiada

Prerrequisitos

Haber leído los bloques 00 y 01. Basta con saber que un dato debe tener significado y que una regla analítica debe poder revisarse.

Práctica

Abre el [notebook de gastos personales](#) o [ejecútalo en Google Colab](#). También puedes resolver el [ejercicio](#). Las [soluciones](#) se consultan al terminar.

Ejecutar código, valores y variables

Objetivos y prerrequisitos

Aprenderás qué hace un programa, cómo ejecutar una celda de notebook y cómo guardar un resultado con un nombre. No necesitas experiencia previa.

Código que transforma valores

Un programa es una lista de instrucciones que transforma información. En un análisis, esa información puede ser el importe de una compra, una fecha o una respuesta de usuario. Un **valor** es una pieza concreta de información: 1200, "web" o True (verdadero).

En Google Colab, un notebook muestra una celda de código y su resultado. Ejecuta primero esto y cambia después un único valor:

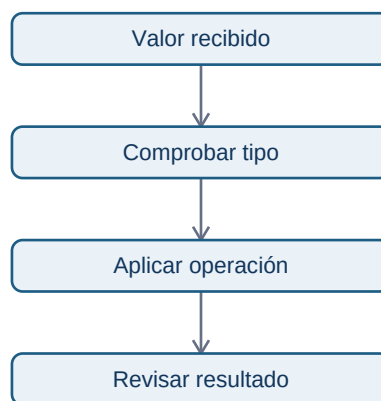
```
ventas = 1200
objetivo = 1500
cumplimiento = ventas / objetivo
print(cumplimiento)
```

Una **variable** es un nombre que referencia un valor. Aquí cumplimiento guarda el resultado 0.8. El signo = no pregunta si dos cosas son iguales: asigna el valor de la derecha al nombre de la izquierda.

Tipos: la forma del dato importa

Python distingue números enteros (3), decimales (3.5), texto ("Madrid"), booleanos (True o False) y ausencia representada por None. El tipo condiciona qué operaciones tienen sentido: sumar 3 + 5 es válido; sumar "3" + "5" un texto y produce "35".

Esta secuencia responde a "¿por qué revisar el tipo antes de calcular?"



El paso de comprobar evita, por ejemplo, tratar un importe escrito como texto como si fuera dinero calculable.

Error habitual

ventas = ventas + 100 parece una igualdad matemática imposible. En programación significa "toma el valor actual de ventas, súmale 100 y guarda el nuevo resultado bajo el mismo nombre". Usarlo sin cuidado puede ocultar el valor original; cuando importe, conserva ambos nombres:

ventas_iniciales y ventas_actualizadas.

Resumen y comprobación

- Una celda ejecuta instrucciones y muestra un resultado.
- Una variable etiqueta un valor; no es una caja mágica independiente.
- El tipo determina qué cálculo es válido.

Prueba `type(ventas)` y `type("1200")`. Explica por qué se diferencian. Continúa con [colecciones](#).

Listas, diccionarios y datos sencillos

Objetivos y prerequisites

Sabrás agrupar varios valores y representar una compra sencilla antes de conocer las tablas de Pandas. Requiere variables y tipos básicos.

Cuando un valor no basta

Una lista guarda una secuencia ordenada. Por ejemplo, los importes de tres pedidos:

```
importes = [12.50, 18.00, 7.20]
primer_importe = importes[0]
```

Los corchetes indican una **lista** y el índice empieza en cero: `importes[0]` es el primer elemento. Esto sorprende al principio, así que no intentes “corregirlo”: compruébalo imprimiendo el resultado.

Un **diccionario** asocia una clave con un valor. Es útil para una observación con campos nombrados:

```
pedido = {"canal": "web", "importe": 42.50, "pagado": True}
print(pedido["importe"])
```

La clave evita depender de una posición. Una lista de diccionarios puede representar varias compras pequeñas; más adelante Pandas convertirá esa estructura en una tabla.

Relación con datos reales

Una API —un mecanismo para que programas intercambien información— suele devolver listas y diccionarios similares. Eso no elimina la necesidad de validar: que un campo se llame `importe` no garantiza que sea numérico, completo o esté expresado en la misma moneda.

Límite y práctica

`pedido["descuento"]` provoca un error si esa clave no existe. No inventes un cero sin preguntar qué significa que falte: podría significar “sin descuento”, “dato desconocido” o “campo no recogido”.

Resume con tus palabras la diferencia entre lista y diccionario y continúa con [condiciones y bucles](#).

Decisiones y repeticiones

Objetivos y prerequisites

Aplicarás una regla distinta según un dato y repetirás una comprobación sobre varios pedidos. Requiere listas y diccionarios.

Condiciones: expresar una regla

Una condición permite que el programa elija. `if` pregunta por una expresión que da `True` o `False`:

```
importe = 120
if importe >= 100:
    categoria = "pedido alto"
else:
    categoria = "pedido habitual"
```

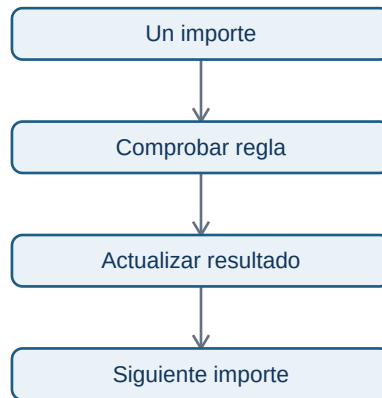
La sangría no es decoración: las líneas desplazadas pertenecen a la rama de la condición. La regla debe coincidir con una definición de negocio. Pregunta si exactamente 100 debe contarse como alto antes de decidir entre `>` y `>=`.

Bucles: repetir sin copiar código

Un bucle `for` visita cada elemento de una colección. El siguiente acumula importes y muestra la idea de agregación que luego hará Pandas:

```
total = 0
for importe in [12.5, 18.0, 7.2]:
    total = total + importe
print(total)
```

Este flujo responde a “¿qué ocurre para cada dato de entrada?”



El programa repite el mismo criterio; no decide de forma inteligente qué regla usar. La calidad de la conclusión depende de que la regla y los datos sean apropiados.

Error habitual

No modifiques una lista mientras la recorres salvo que entiendas las consecuencias: puedes saltarte elementos. Para aprender, crea una lista nueva para los resultados o revisa primero el tamaño y contenido de la original.

Resumen

Las condiciones expresan criterios y los bucles los aplican repetidamente. Sigue con [funciones](#).

Funciones y alcance

Objetivos y prerequisites

Aprenderás a encapsular una regla reutilizable, diferenciar entrada y salida y evitar depender de variables ocultas. Requiere condiciones y bucles.

Dar nombre a una transformación

Una **función** es un fragmento de código con un nombre, entradas y una salida. Evita copiar la misma regla en diez lugares y hace que el análisis se pueda revisar por partes.

```
def clasificar_importe(importe):  
    if importe >= 100:
```

```
        return "alto"
    return "habitual"

clasificar_importe(120)
```

importe es un **parámetro**: el nombre que la función usa para recibir un dato. `return` devuelve un resultado. Una función útil responde una pregunta concreta: “dado un importe, ¿qué etiqueta corresponde según esta regla?”.

Alcance: qué nombres existen dónde

Los nombres creados dentro de una función normalmente solo existen durante esa llamada. Esto se llama **alcance**. Es una protección: una función debería depender de sus entradas, no de una variable lejana cuyo valor puede cambiar sin avisar.

```
limite = 100

def es_alto(importe, limite):
    return importe >= limite
```

Aunque parece repetitivo pasar `limite`, deja visible el supuesto. En un análisis, los supuestos invisibles son difíciles de auditar.

Caso IT y límite

Una función puede estandarizar una comprobación de eventos: clasificar una duración de carga como lenta o aceptable. Pero no convierte el umbral en verdad universal: 2 segundos puede ser aceptable en una página y grave durante un pago. Documenta de dónde sale el umbral.

Resumen y práctica

- Las funciones nombran transformaciones repetibles.
- Parámetros hacen visibles las entradas; `return` entrega la salida.
- Evitar dependencias ocultas facilita revisar y probar.

Reescribe el cálculo de “pedido alto” como función y pruébalo con 99, 100 y 101. Después estudia [errores y depuración](#).

Errores y depuración

Objetivos y prerequisites

Sabrás leer el mensaje de error, formular una hipótesis mínima y comprobarla sin cambiar diez cosas a la vez. Requiere haber ejecutado código sencillo.

Un error también es evidencia

Cuando Python no puede continuar, muestra un mensaje con una última línea relevante. `NameError` suele indicar que un nombre no existe; `TypeError`, que se intentó una operación incompatible; `KeyError`, que falta una clave del diccionario. El mensaje no sustituye la comprensión, pero delimita qué revisar.

Ejemplo:

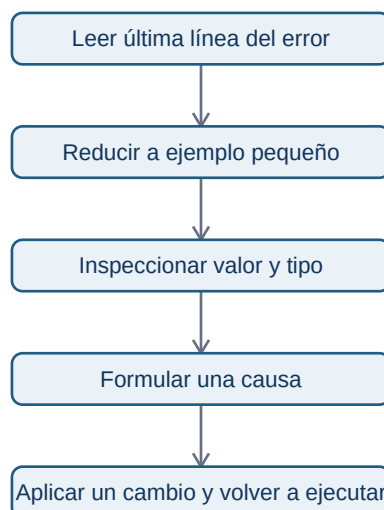
```
importe = "42.50"  
importe + 10
```

El problema no es que Python “falle”: `importe` contiene texto, no un número. La corrección solo debe hacerse si la regla de origen confirma que ese texto representa un importe válido:

```
importe_numerico = float(importe)
```

Método de depuración

Este flujo responde a “¿cómo investigar sin adivinar?”



Reducir el ejemplo evita mezclar varios problemas. Usa `print(valor)` y `type(valor)` antes de cambiar código. Si la causa es un dato inesperado, no la tapes con una conversión automática: registra cuántos casos hay y decide qué significan.

Error habitual: silenciar en lugar de entender

`try/except` permite manejar errores, pero capturarlos todos y continuar puede ocultar importes inválidos o pedidos perdidos. Úsalo para casos esperados y registra qué ocurrió. Un análisis correcto que descarta silenciosamente el 20 % de los datos no es fiable.

Resumen

Depurar es un proceso de evidencia: leer, aislar, inspeccionar, probar. Continúa con [estilo y práctica](#).

Estilo y práctica guiada

Objetivos y prerequisites

Aplicarás las piezas del bloque en un problema pequeño y aprenderás a escribir código que otra persona pueda leer. Requiere todas las lecciones anteriores.

Legibilidad es una propiedad analítica

Un programa de análisis no se evalúa solo por “funciona hoy”. Debe dejar claro qué representa cada valor y qué regla se aplicó. Usa nombres como `gasto_por_categoria`, no `x`; evita repetir números mágicos como 100 sin explicar su significado; separa carga de datos, transformación y resultado.

Un ejemplo legible:

```
LIMITE_REVISAR = 100

def categorias_sobre_limite(movimientos, limite):
    total_por_categoria = {}
    for movimiento in movimientos:
        categoria = movimiento["categoria"]
        importe = movimiento["importe"]
        total_por_categoria[categoria] = total_por_categoria.get(categoria, 0) + importe
    return [categoria for categoria, total in total_por_categoria.items() if total > limite]
```

Antes de reutilizarlo en una empresa, define cómo se tratan devoluciones, moneda, movimientos duplicados y valores ausentes. El código implementa una decisión; no decide por ti qué es correcto.

Comprobaciones mínimas

Prueba casos normales y casos límite: lista vacía, importe exactamente 100, importe negativo y un texto donde debería haber número. Escribir esos ejemplos antes de confiar en el resultado es una

forma simple de prueba.

Ejercicio de cierre

Resuelve la [práctica de gastos](#) y compara tu razonamiento con las [soluciones](#) solo al terminar. Si solo tienes móvil, escribe primero el pseudocódigo en texto: entradas, pasos y salidas.

Puente al siguiente bloque

Python permite operar con estructuras pequeñas. En NumPy y Pandas aplicarás operaciones parecidas a miles de valores y tablas, pero conservarás las mismas preguntas: ¿qué representa cada dato?, ¿qué regla se aplica?, ¿cómo se comprueba?

Bloque 03 - Matemáticas aplicadas al análisis

Propósito y ruta adaptable

Conectar herramientas matemáticas con decisiones de negocio y análisis. Si manejas porcentajes, medias ponderadas, funciones y vectores, usa las explicaciones generales como repaso; las aplicaciones, los límites y la interpretación no son opcionales.

Lecciones

1. [Magnitudes, porcentajes y tasas](#)
2. [Promedios, ponderación y agregación](#)
3. [Funciones, vectores y matrices](#)
4. [Tiempo, crecimiento y comparaciones](#)

Resultado esperado

Podrás calcular y comunicar un cambio sin mezclar unidades, elegir una agregación defendible y cuestionar comparaciones aparentemente obvias.

Magnitudes, porcentajes y tasas

Objetivos y prerequisites

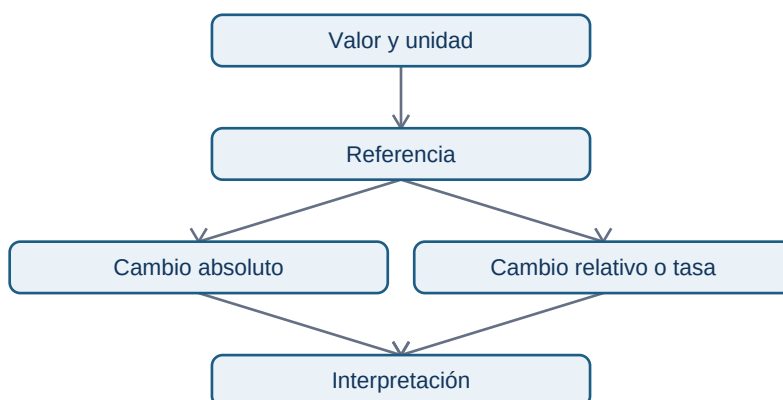
Sabrás separar cambio absoluto, cambio relativo y puntos porcentuales. La aritmética básica es suficiente; si ya dominas fórmulas, céntrate en los ejemplos y errores de interpretación.

Una cifra necesita unidad y referencia

Pasar de 100 a 120 pedidos supone un cambio absoluto de 20 pedidos y un crecimiento relativo del 20 %: $(120 - 100) / 100$. Ambas cifras son correctas, pero responden preguntas distintas. El cambio absoluto ayuda a estimar capacidad; el relativo permite comparar grupos de tamaño distinto.

Una tasa relaciona cantidades: por ejemplo, 30 compras de 1 000 visitas son una tasa de conversión del 3 %. No digas “subió un 2 %” si pasó de 3 % a 5 %: aumentó **2 puntos porcentuales** y aproximadamente un 66,7 % relativo. Esa diferencia cambia la percepción de impacto.

Este recorrido responde a “¿qué debe declararse antes de comparar un número?”



La referencia puede ser ayer, el objetivo, otro segmento o el mismo mes del año anterior; elegirla es una decisión analítica, no una operación automática.

Ejemplo y contraejemplo

Una app pasa de 10 a 20 conversiones: +10 conversiones y +100 %. Otra pasa de 10 000 a 10 100: +100 conversiones pero +1 %. Presentar solo porcentaje hace enorme el primer cambio; presentar solo conteo oculta que el segundo afecta a más clientes. Comunica ambos cuando importen.

Un descenso del 20 % después de un aumento del 20 % no vuelve al inicio: $100 \times 1,2 \times 0,8 = 96$. Los porcentajes se aplican a bases distintas.

Resumen y práctica

- Declara unidad, población y referencia.
- Distingue porcentaje, tasa y punto porcentual.
- El tamaño de la base cambia la interpretación.

Calcula el cambio absoluto, relativo y en puntos porcentuales si una conversión pasa de 4 % a 5 %. Luego sigue con [ponderación](#).

Promedios, ponderación y agregación

Objetivos y prerequisites

Aprenderás cuándo un promedio resume un conjunto y cuándo lo distorsiona. Requiere comprender porcentajes y tasas.

Un promedio siempre combina observaciones

La media aritmética suma valores y divide por su número. Es útil para importes comparables, pero una media de tasas puede ser engañosa si cada grupo tiene un tamaño distinto. Si el país A convierte 8 de 10 visitas (80 %) y B convierte 1 000 de 10 000 (10 %), la media simple de 45 % no describe la conversión conjunta. La respuesta correcta suma éxitos y oportunidades: $1008 / 10010$, aproximadamente 10,1 %.

Eso es una **media ponderada**: cada tasa pesa según su denominador. No es un detalle de fórmula; evita tomar decisiones de inversión basadas en segmentos pequeños y extremos.

Agregar cambia el grano

Antes de sumar o promediar, pregunta qué representa una fila. Si cada fila es un pedido, sumar importes da ingresos por pedido. Si cada fila es un usuario mensual, sumar ingresos puede duplicar clientes que compraron varias veces. El nivel al que se describe un dato se llama **grano** y se estudió en el bloque 01.

Error habitual: promedio de promedios

Un dashboard muestra conversión diaria y calcula la media de siete porcentajes. Puede ser válido si cada día tiene el mismo tráfico; si no, conviene dividir compras totales entre visitas totales. Conserva numerador y denominador: permiten revisar y reponderar.

Resumen

El promedio no es neutral: depende de qué observaciones se incluyan y cuánto pesa cada una. Continúa con [funciones](#), [vectores](#) y [matrices](#).

Funciones, vectores y matrices

Objetivos y prerequisites

Comprenderás las ideas matemáticas que harán legibles NumPy y Pandas: una función transforma entradas; un vector reúne valores; una matriz organiza muchos vectores.

Transformar entradas en salidas

Una **función** expresa una relación: dado número de clientes y precio, devuelve ingresos. $\text{ingresos}(\text{clientes}, \text{precio}) = \text{clientes} \times \text{precio}$. No afirma que ambas variables sean independientes ni que subir precio no cambie clientes: solo define el cálculo bajo unos supuestos.

Un **vector** es una lista ordenada de valores de la misma clase, por ejemplo las ventas de tres días: $[120, 140, 110]$. Una **matriz** organiza muchas filas de valores: cada fila podría ser un día y cada columna pedidos, ingresos y devoluciones. En código, un array de NumPy permitirá aplicar un cálculo a todos los elementos sin escribir un bucle manual.

Relación con el análisis

Los objetos matemáticos no sustituyen el significado. Dos columnas con mil números pueden formar una matriz, pero no por ello es razonable sumarlas si una contiene euros y otra minutos. La estructura permite calcular; el contexto decide si el cálculo responde una pregunta válida.

Resumen y puente

Funciones hacen explícitas transformaciones; vectores y matrices permiten representar muchas observaciones. En el bloque 04 aprenderás a manejarlos eficientemente con NumPy.

Tiempo, crecimiento y comparaciones

Objetivos y prerequisites

Sabrás elegir una comparación temporal defendible y distinguir nivel, variación y crecimiento acumulado. Requiere porcentajes.

El tiempo no siempre avanza de forma comparable

Comparar junio con mayo puede ser útil, pero una tienda de viajes puede tener una estacionalidad fuerte: junio suele diferir de mayo por razones repetidas cada año. Compara también con junio del año anterior y con una referencia acordada. La elección depende de la decisión: planificación anual, respuesta a una incidencia o seguimiento semanal.

El crecimiento compuesto encadena cambios: si una métrica crece 10 % dos meses, el factor es $1,1 \times 1,1 = 1,21$, no 1,20. Para comunicarlo, muestra periodo inicial, final y fórmula, no una etiqueta vaga de “crecimiento”.

Límite: una serie no demuestra causalidad

Que una métrica cambie tras una acción no identifica por sí mismo la causa. El tiempo puede coincidir con campañas, festivos, cambios de mercado o problemas de medición. Las comparaciones temporales son evidencia descriptiva que requiere contexto, segmentos y, cuando sea necesario, experimentos.

Cierre del bloque

Las matemáticas aplicadas sirven para no confundir escalas, promedios ni referencias. Antes de automatizar con NumPy o Pandas, pregunta siempre: ¿qué mide este número, sobre qué población y frente a qué comparación?

Aplica el bloque en la [práctica de tasas y promedios](#) antes de leer su solución.

Bloque 04 - NumPy y cálculo vectorizado

Propósito

Usar arrays para representar datos numéricos y aplicar cálculos sobre colecciones completas. NumPy prepara el modelo mental de Pandas, pero no reemplaza la comprensión del significado de cada número.

Lecciones

1. Arrays y cálculo vectorizado
2. Selección, máscaras y forma
3. Broadcasting, simulación y reproducibilidad

Arrays y cálculo vectorizado

Objetivos y prerequisites

Sabrás crear un array numérico y aplicar un cálculo a todos sus elementos. Requiere entender listas y vectores de los bloques 02 y 03.

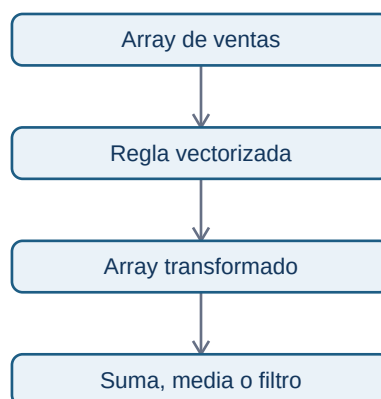
De una lista a un array

Un **array** es una estructura para valores organizados, normalmente del mismo tipo, que permite operaciones numéricas eficientes. En Python se importa NumPy con un alias convencional:

```
import numpy as np
ventas = np.array([120, 140, 110])
ventas_con_iva = ventas * 1.21
```

La última línea no multiplica la lista como texto ni requiere un `for`: aplica la operación elemento a elemento. Eso se llama **vectorización**. Expresa “aplica la misma regla a cada venta”, una intención fácil de revisar.

Este flujo responde a “¿qué ocurre cuando la regla llega a una colección completa?”



Límite analítico

Que una operación sea rápida no la hace correcta. Multiplicar por 1,21 solo procede si todos los valores comparten moneda, representan importes sin IVA y la regla de negocio aplica a todos. Vectorizar una mala regla propaga el error más deprisa.

Resumen

Un array reúne números; la vectorización aplica una operación a cada uno. En la siguiente lección seleccionarás subconjuntos sin perder de vista el criterio usado.

Selección, máscaras y forma

Objetivos y prerequisites

Aprenderás a elegir elementos por posición o condición y a interpretar la forma de un array. Requiere arrays básicos.

Preguntar una condición a cada elemento

Una **máscara booleana** contiene True o False para cada posición. Es la traducción de una pregunta: “¿esta venta supera 100?”

```
ventas = np.array([80, 125, 210])
es_alta = ventas > 100
ventas[es_alta] # array([125, 210])
```

La máscara es valiosa porque hace visible el criterio. Antes de filtrar pedidos “altos”, define por qué 100 es el límite y revisa cuántos elementos quedan fuera.

La **forma** (shape) describe dimensiones. Un array de tres ventas tiene forma (3,); una matriz de dos días y tres métricas puede tener (2, 3). La forma no da significado a filas y columnas: debes documentarlo.

Error habitual

Una máscara de longitud distinta al array no se puede aplicar correctamente. Más grave aún: una máscara correcta técnicamente puede estar desalineada conceptualmente si proviene de otro periodo o de clientes ordenados de forma distinta.

Resumen

Seleccionar es formular un criterio. Comprueba siempre forma, orden y significado antes de combinar arrays.

Broadcasting, simulación y reproducibilidad

Objetivos y prerequisites

Comprenderás cómo NumPy combina dimensiones compatibles y por qué una simulación debe poder repetirse. Requiere forma y arrays.

Broadcasting sin magia

Broadcasting es la regla por la que NumPy aplica un vector compatible a una matriz sin copiarlo manualmente. Si una matriz contiene ventas por día y canal, restar la media de cada canal puede centrar sus columnas. No memorices casos: imprime shape y comprueba qué dimensión representa cada valor antes de operar.

Simular para explorar, no para fabricar evidencia

Puedes generar números aleatorios para practicar o evaluar un método. Una **semilla** fija el punto de partida del generador y permite repetir el mismo ejemplo:

```
generador = np.random.default_rng(42)
muestra = generador.normal(loc=100, scale=15, size=5)
```

El 42 no hace la simulación más verdadera; hace el resultado reproducible. Una muestra simulada sirve para aprender o estudiar escenarios, no para afirmar qué hicieron clientes reales.

Cierre y ejercicio

NumPy permite transformar, seleccionar y resumir números de forma concisa. Antes de Pandas, resuelve una práctica: crea ventas, aplica una máscara y explica qué supuesto contiene el umbral. Documenta semilla y forma. El mismo criterio se aplicará a tablas reales.

Bloque 05 - Pandas: manipulación de datos

Propósito

Preparar datos tabulares con trazabilidad. Pandas no “arregla” datos por sí solo: permite expresar y validar decisiones sobre ellos.

Lecciones

1. DataFrames, importación y perfilado
2. Selección, tipos y limpieza
3. Columnas derivadas y agregaciones
4. Uniones y cardinalidad
5. Validación y trazabilidad
6. Caso integrado de pedidos

Práctica

Resuelve [la limpieza de pedidos](#) y después consulta [la solución razonada](#).

DataFrames, importación y perfilado

Objetivos y prerequisites

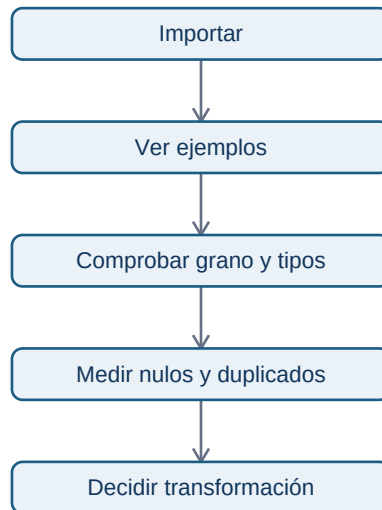
Sabrás abrir una tabla con Pandas e inspeccionarla antes de modificarla. Requiere los bloques de datos y Python.

Un **DataFrame** es una tabla en memoria: filas (observaciones) y columnas (variables) con nombres. Un archivo CSV puede guardar una tabla, pero abrirlo no garantiza que cada columna tenga el tipo ni el significado esperado.

```
import pandas as pd
pedidos = pd.read_csv("pedidos.csv")
pedidos.head()
pedidos.info()
```

`head()` muestra ejemplos; `info()` enseña número de filas, columnas, tipos y valores no nulos. Complementalos con `describe()`, revisión de categorías y comprobación del grano: ¿una fila representa un pedido, una línea de pedido o un usuario?

Este flujo responde a “¿qué debe ocurrir antes de calcular?”



Un error habitual es llamar “ventas” a una columna sin verificar moneda, impuestos o devoluciones. El perfilado abre preguntas; no las responde automáticamente.

Sigue con [selección](#), [tipos](#) y [limpieza](#).

Selección, tipos y limpieza

Objetivos y prerequisites

Seleccionarás columnas y filas, convertirás tipos explícitamente y tratarás problemas sin borrar información a ciegas.

Seleccionar una columna responde una pregunta concreta: `pedidos["importe"]`. Filtrar filas aplica un criterio visible: `pedidos[pedidos["estado"] == "pagado"]`. Antes de filtrar, cuenta qué se excluye y por qué; “pagado” puede ser una definición distinta a “pedido creado”.

Los valores importados como texto requieren conversión controlada:

```
pedidos["importe"] = pd.to_numeric(pedidos["importe"], errors="coerce")
pedidos["fecha"] = pd.to_datetime(pedidos["fecha"], errors="coerce")
```

`coerce` convierte valores inválidos en ausentes. Es útil porque no inventa una cifra, pero obliga a medir y decidir qué hacer con esos ausentes. No elimines nulos por costumbre: pueden concentrarse en un canal y sesgar el resultado.

Resumen

Limpiar es convertir una regla de calidad en código verificable. Continúa con [transformación y agregación](#).

Columnas derivadas y agregaciones

Objetivos y prerequisites

Crearás medidas derivadas y resumirás una tabla sin perder de vista el grano.

Una columna derivada expresa una regla: `importe_netto = importe - descuento`. Documenta si el descuento ya incluye impuestos y qué sucede cuando falta. `groupby` agrupa filas y aplica una agregación:

```
ventas_canal = pedidos.groupby("canal", as_index=False).agg(
    pedidos=("pedido_id", "nunique"),
    ingresos=("importe_netto", "sum")
)
```

`nunique` cuenta identificadores únicos; `count` cuenta valores no nulos. Elegir uno u otro cambia la métrica. Un promedio de importe también puede ocultar la distribución, por lo que conviene acompañarlo de volumen y percentiles cuando la decisión lo requiera.

Límite

Agregar convierte muchas filas en pocas. Es útil para comparar canales, pero puede ocultar diferencias por país, dispositivo o periodo. Conserva la tabla de origen y registra cada agregación.

Sigue con [uniones y cardinalidad](#).

Uniones y cardinalidad

Objetivos y prerequisites

Combinarás tablas comprobando qué clave conecta las filas y cuántas coincidencias son válidas.

Una unión (`merge`) cruza dos tablas mediante una **clave**, por ejemplo `cliente_id`. Antes de ejecutarla declara la cardinalidad: uno a uno, uno a muchos o muchos a uno. Muchos a muchos puede ser correcto, pero multiplica combinaciones y requiere una justificación explícita.

```
pedidos_con_clientes = pedidos.merge(
    clientes, on="cliente_id", how="left", validate="many_to_one"
)
```

`validate="many_to_one"` convierte un supuesto en una comprobación: muchos pedidos pueden corresponder a un cliente, pero cada pedido no debe encontrar dos fichas de cliente. Tras unir, compara filas, claves sin coincidencia y totales monetarios.

Error habitual

Ver una columna nueva y asumir que la unión funcionó. Si `clientes` tiene duplicados por error, cada pedido puede repetirse y los ingresos se inflan. La sintaxis válida no demuestra una relación válida.

Continúa con [validación y trazabilidad](#).

Validación y trazabilidad

Objetivos y prerequisites

Definirás controles simples que convierten una transformación en un paso revisable.

Tras cada paso relevante guarda observaciones: número de filas, claves únicas, porcentaje de nulos y totales de negocio. Una validación puede ser una aserción:

```
assert pedidos["pedido_id"].is_unique
assert pedidos["importe_netto"].notna().all()
```

No uses una aserción para ocultar un problema. Si falla, inspecciona los registros y decide si el supuesto era incorrecto o si hay un defecto de datos. Registra filtros, versión de la fuente y fecha de extracción: ese rastro permite reproducir el análisis.

Resumen

Validar no es un último adorno; acompaña a cada transformación. Aplica ahora todo el ciclo en el [caso integrado](#).

Caso integrado de pedidos

Objetivos y prerequisites

Usarás el flujo completo para responder una pregunta simple: “¿qué canal aporta ingresos netos y cuántos pedidos válidos hay?”.

Primero formula el contrato: una fila es un pedido; solo se incluyen estados pagados; importe neto excluye descuento; el periodo es el mes analizado. Después perfila, convierte tipos, mide registros inválidos, crea la columna neta y agrupa por canal. Finalmente compara el total agrupado con el total de pedidos filtrados.

El resultado debe incluir una limitación: si hay pedidos devueltos después de la extracción, los ingresos no representan todavía margen final. Ese tipo de frase es parte del análisis, no una excusa.

Resuelve la [limpieza de pedidos](#) y revisa la solución razonada. El siguiente bloque explorará lo que estos resúmenes sugieren, sin convertirlos de inmediato en causalidad.

Bloque 06 - Análisis exploratorio de datos

Propósito

Explorar datos para descubrir patrones, anomalías y preguntas nuevas sin confundir exploración con demostración causal.

Lecciones

1. Preguntas y perfil exploratorio
2. Distribuciones, segmentos y valores extremos
3. Relaciones, correlación y explicaciones rivales
4. Registro de hallazgos y decisiones

Práctica

Resuelve [la investigación de una caída](#) antes de mirar [la guía de solución](#).

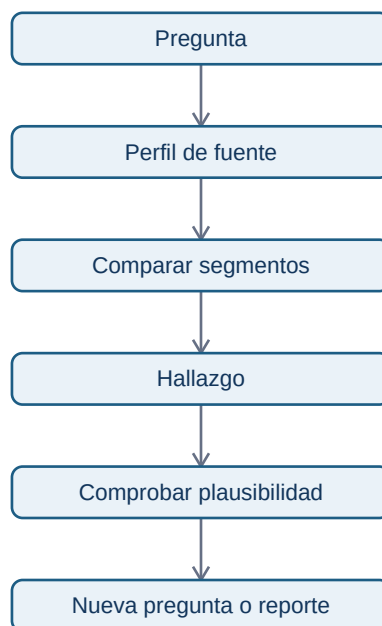
Preguntas y perfil exploratorio

Objetivos y prerequisites

Convertirás un conjunto de datos en preguntas de exploración y comprobarás si la fuente puede responderlas. Requiere manejo básico de Pandas.

El análisis exploratorio, o **EDA**, es una investigación abierta pero disciplinada. No empieza con “haz todos los gráficos”; empieza con una pregunta como “¿en qué segmento se concentra la caída de

pedidos?" y con un perfil de grano, periodo, cobertura, nulos y duplicados.



El hallazgo genera una hipótesis, no un veredicto. Si faltan datos de dispositivo, no concluyas que no importa: concluye que la fuente no permite evaluarlo.

Resumen

El EDA limita el espacio de dudas con evidencia visible. Sigue con [distribuciones y segmentos](#).

Distribuciones, segmentos y valores extremos

Objetivos y prerequisites

Interpretarás cómo se reparte una medida y decidirás cuándo investigar valores extremos en vez de eliminarlos.

Una **distribución** muestra cómo se repiten los valores. Además de una media, observa mediana, dispersión, asimetría y percentiles. En importes de pedido, unos pocos pedidos empresariales pueden elevar la media aunque la mayoría de clientes gaste poco.

Segmentar divide observaciones por una característica relevante: canal, país, dispositivo o cohorte. Una tendencia global puede esconder un canal en caída y otro en crecimiento. El segmento se elige por una hipótesis de negocio, no porque haya columnas disponibles.

Un **outlier** es un valor inusual respecto al resto; no es sinónimo de error. Puede ser una compra fraudulenta, un cliente clave, una moneda mal parseada o una unidad distinta. Conserva la evidencia, clasifica la causa y documenta la regla si decides excluirlo.

Resumen

Describe la distribución antes de resumirla y explica cualquier exclusión. Continúa con [relaciones y causalidad](#).

Relaciones, correlación y explicaciones rivales

Objetivos y prerequisites

Distinguirás asociación observada, explicación posible y evidencia causal.

Dos medidas pueden moverse juntas. Una **correlación** resume asociación lineal, pero no identifica por qué ocurre. Las visitas y las ventas pueden subir por una campaña; el precio y las devoluciones pueden variar porque se venden productos distintos. Una tercera variable, un cambio de medición o puro azar pueden explicar la relación.

Antes de recomendar una acción, formula explicaciones rivales y busca qué dato las distinguiría. Un experimento o un diseño causal puede aportar mejor evidencia; el EDA prepara esa investigación.

Error habitual

Ordenar una tabla por dos columnas y afirmar una causa porque el patrón “parece claro”. La visualización ayuda a detectar preguntas; no elimina confusión, sesgo de selección ni estacionalidad.

Resumen

Una asociación es un punto de partida. Registra qué observaste y qué sería necesario para sostener una explicación.

Registro de hallazgos y decisiones

Objetivos y prerequisites

Aprenderás a convertir una exploración en un artefacto revisable por producto, ingeniería o dirección.

Para cada hallazgo anota pregunta, fuente y periodo, filtros, método, resultado, interpretación, límites y siguiente acción. Un ejemplo: "La conversión móvil cayó 1,8 puntos desde la versión 4.2; el patrón aparece en Android y no en web; falta comprobar cambios de tracking y errores del formulario".

No escribas "la versión causó la caída" si solo existe coincidencia temporal. La precisión del lenguaje protege al equipo de decidir demasiado pronto.

Resuelve la [investigación de una caída](#). En el bloque siguiente aprenderás a elegir gráficos que hagan visible esta evidencia sin manipular la percepción.

Bloque 07 - Visualización y comunicación

Propósito

Elegir y construir visualizaciones que permitan comprender una decisión con rapidez, sin distorsionar los datos.

Lecciones

1. [De la pregunta al tipo de gráfico](#)
2. [Diseño honesto y accesible](#)
3. [De exploración a comunicación](#)
4. [Dashboards y entregables profesionales](#)

Ejercicio

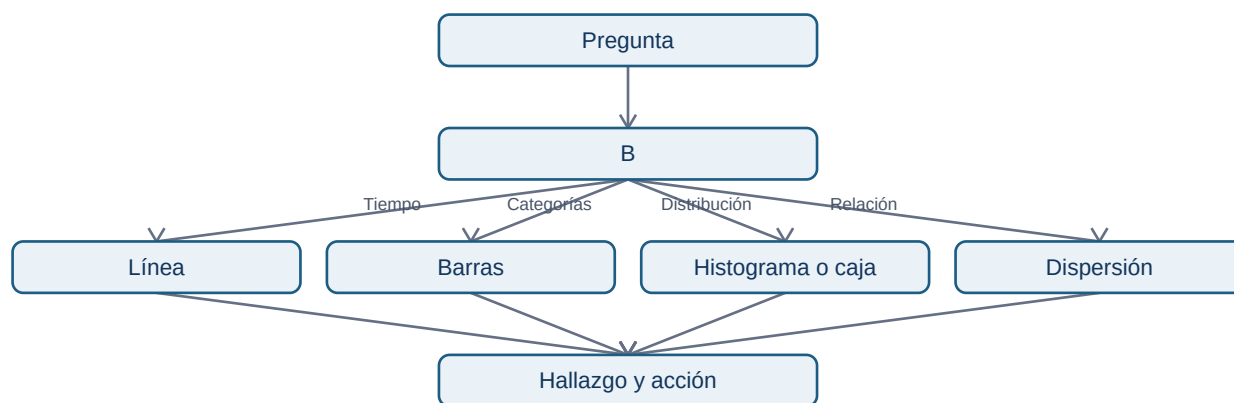
Haz el [diagnóstico de gráficos](#) y comprueba [los criterios](#).

De la pregunta al tipo de gráfico

Objetivos y prerequisites

Sabrás escoger una representación según la comparación que una decisión necesita. Requiere EDA básico.

Un gráfico no es una decoración de una tabla: es una forma de hacer visible una comparación. Pregunta primero si necesitas mostrar evolución, diferencias entre categorías, distribución o relación entre dos medidas.



Una línea responde bien a “¿cómo cambió semanalmente la conversión?”. Unas barras ordenadas responden mejor a “¿qué canal tiene más pedidos?”. Un histograma muestra si un promedio es representativo. Un gráfico de dispersión ayuda a explorar asociación, no a afirmar causalidad.

Error habitual

Elegir un gráfico porque “queda profesional”. Un gráfico circular con muchas categorías impide comparar; una línea sobre categorías sin orden temporal inventa continuidad. El gráfico correcto depende de la pregunta y del tipo de dato.

Resumen

Declara la pregunta antes del gráfico. Continúa con [diseño honesto](#).

Diseño honesto y accesible

Objetivos y prerequisites

Aprenderás a etiquetar, escalar y colorear un gráfico sin exagerar diferencias ni excluir a parte de la audiencia.

Un gráfico honesto declara unidad, periodo, población y fuente cuando son necesarios. En barras que comparan magnitudes, empezar el eje en cero evita que diferencias pequeñas parezcan enormes. En líneas puede usarse otro rango si se explica y se busca estudiar variación, no tamaño absoluto.

El color debe codificar una diferencia con significado: versión A frente a B, cumplimiento frente a riesgo. No hagas que el único mensaje dependa de rojo y verde; añade etiquetas, contraste y patrones si el gráfico se va a reutilizar.

Límite

La claridad no significa ocultar incertidumbre: si un valor procede de pocos usuarios o una estimación, muestra contexto, intervalo o nota metodológica. Simplificar es retirar ruido, no retirar condiciones que cambiarían una decisión.

Resumen

Cada elección visual transmite una interpretación. Verifica escalas, etiquetas y accesibilidad antes de presentar.

De exploración a comunicación

Objetivos y prerequisites

Separarás un gráfico para investigar de uno para recomendar una decisión.

Durante exploración puedes producir muchos gráficos, probar segmentos y descubrir errores. Un gráfico explicativo reduce esa exploración a una afirmación revisable: título con hallazgo, comparación destacada, definición de métrica y límite relevante.

Por ejemplo, “La conversión móvil cayó 1,8 puntos desde la versión 4.2” es más informativo que “Conversión por semana”. Acompáñalo de población, ventana temporal y una nota: “asociación temporal; pendiente validar cambio de tracking”.

Error habitual

Convertir el dashboard exploratorio en una diapositiva ejecutiva con veinte series y filtros. El receptor no puede saber qué mirar ni qué acción se propone. Una buena narrativa deja clara evidencia, recomendación y grado de confianza.

Práctica

Reformula el título de un gráfico que solo diga “Ventas mensuales” y explica qué dato necesitas para justificar el nuevo mensaje.

Dashboards y entregables profesionales

Objetivos y prerequisites

Diseñarás un entregable que responda a una decisión y no solo muestre indicadores.

Un **dashboard** reúne métricas para seguimiento continuo; no sustituye un análisis cuando hay una decisión nueva. Cada panel debe indicar definición, actualización, propietario y acción esperada. Una presentación o ticket de Jira puede ser mejor para explicar una incidencia concreta: contexto, evidencia, recomendación, riesgos y siguiente paso.

Antes de entregar pregunta: ¿quién actúa?, ¿qué cambiaría si el dato varía?, ¿qué limitación debe conocer? Si no hay respuesta, probablemente sobra un gráfico o falta una pregunta.

Completa el [diagnóstico de gráficos](#). El bloque siguiente añadirá la incertidumbre que una visualización por sí sola no puede resolver.

Bloque 08 - Estadística para decisiones

Propósito

Medir incertidumbre, evaluar diferencias y comunicar resultados sin convertir una prueba estadística en una respuesta automática.

Lecciones

1. [Describir variabilidad](#)
2. [Población, muestra y sesgo](#)
3. [Probabilidad e incertidumbre](#)
4. [Intervalos y pruebas de hipótesis](#)
5. [Experimentos A/B](#)
6. [Tamaño de efecto y decisión](#)

Práctica

Analiza [un experimento de onboarding](#) y revisa [la interpretación](#).

Describir variabilidad

Objetivos y prerequisites

Sabrás resumir un conjunto sin esconder su dispersión. Requiere promedios y distribuciones.

La media responde “¿cuál es el promedio?”, pero no “¿cuánto varían los casos?”. Mediana, percentiles, rango y desviación estándar describen perspectivas distintas. En tiempos de carga, una media de dos segundos puede convivir con usuarios que esperan veinte; los percentiles altos suelen importar para experiencia real.

No elijas la medida que haga mejor la historia. Declara por qué la métrica representa la decisión: media para coste total esperado, mediana para cliente típico, percentil 95 para un compromiso de rendimiento.

Resumen

Centro y variabilidad se interpretan juntos. Continúa con [población y muestra](#).

Población, muestra y sesgo

Objetivos y prerequisites

Distinguirás el conjunto sobre el que quieres decidir de los datos que realmente observaste.

La **población** es el conjunto de interés, por ejemplo todos los nuevos usuarios elegibles. Una **muestra** es una parte observada. Un **estadístico** resume la muestra; un parámetro describe la población. Muestras diferentes producen resultados diferentes: esa variabilidad no es un fallo, es la razón para comunicar incertidumbre.

El problema no se arregla solo con más filas. Si solo respondieron usuarios muy activos a una encuesta, hay **sesgo de selección**: la muestra puede ser grande y seguir sin representar a la población.

Resumen

Pregunta siempre quién quedó dentro, quién fuera y por qué. Sigue con [probabilidad e incertidumbre](#).

Probabilidad e incertidumbre

Objetivos y prerequisites

Usarás probabilidad como modelo de incertidumbre, no como promesa sobre un caso individual.

Una probabilidad expresa qué tan frecuente sería un resultado dentro de un modelo y unas condiciones. Si una conversión histórica es 10 %, no significa que cada décimo usuario vaya a comprar ni que el siguiente tenga 10 % “garantizado”. Depende de población, periodo, medición y estabilidad del proceso.

La incertidumbre aparece porque observamos una parte del proceso, existe variación y las mediciones pueden tener error. Separar esos componentes evita comunicar una cifra estimada como si fuera exacta.

Resumen

Un modelo probabilístico necesita supuestos explícitos. Sigue con [intervalos y pruebas](#).

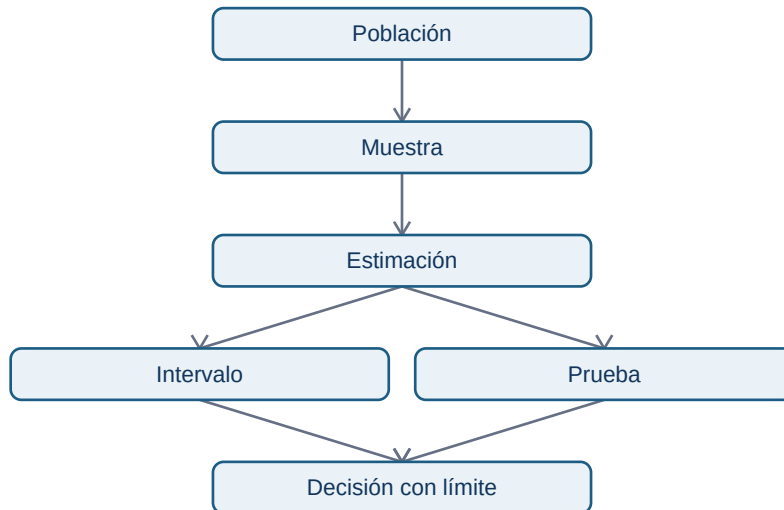
Intervalos y pruebas de hipótesis

Objetivos y prerequisites

Interpretarás un intervalo y un p-valor sin atribuirles un significado que no tienen.

Un intervalo de confianza ofrece un rango de valores compatibles con un método, datos y nivel de confianza bajo sus supuestos. Una prueba compara datos con una **hipótesis nula**, por ejemplo “no hay diferencia de conversión”. Un p-valor pequeño indica que los datos serían poco compatibles con esa hipótesis si el modelo fuera correcto.

No dice la probabilidad de que la hipótesis nula sea cierta, no mide importancia de negocio y no corrige sesgo, medición mala ni pruebas repetidas. Comunica efecto absoluto, relativo, intervalo y decisión propuesta.



Resumen

La inferencia cuantifica incertidumbre; no sustituye juicio ni diseño. Continúa con [experimentos A/B](#).

Experimentos A/B

Objetivos y prerequisites

Diseñarás la estructura mínima de un experimento antes de mirar su resultado.

Un experimento A/B asigna unidades elegibles a variantes para comparar una métrica. Define antes población, unidad de asignación, métrica principal, guardrails, duración, tamaño necesario y criterio de decisión. Si aleatorizas por usuario pero mides por sesión sin cuidado, puedes contar varias veces una experiencia.

No mires cada día hasta encontrar significación: ese comportamiento eleva falsos positivos. También vigila calidad del tracking, exposición real a la variante y efectos por segmentos relevantes.

Límite

Un experimento válido estima efecto en la población y periodo estudiados; no garantiza el mismo efecto después de lanzar globalmente ni en otro mercado.

Práctica

Analiza el [experimento de onboarding](#) antes de leer su solución.

Tamaño de efecto y decisión

Objetivos y prerequisites

Vincularás una diferencia estadística con coste, beneficio, riesgo y recomendación.

Una diferencia minúscula puede resultar “significativa” con millones de usuarios y aun así no pagar el coste de ingeniería. Una diferencia grande con pocos datos puede ser prometedora pero incierta. Por eso una decisión profesional reúne tamaño absoluto, cambio relativo, intervalo, volumen afectado, guardrails y reversibilidad de la acción.

Ejemplo: +0,2 puntos de conversión puede significar miles de pedidos si hay mucho tráfico, pero no conviene lanzar si aumenta reclamaciones o si el intervalo incluye una pérdida importante. La recomendación debe declarar el umbral que hace que actuar merezca la pena.

Cierre

Estadística no concede permiso automático para afirmar o lanzar. Ayuda a calibrar qué se sabe, qué riesgo queda y qué evidencia faltaría.

Bloque 09 - SQL, NoSQL y almacenamiento

Propósito

Entender cómo viven los datos en una empresa, consultar tablas con SQL y saber cuándo un modelo documental o clave-valor exige otro diseño.

Lecciones

1. [Modelo relacional, tablas y grano](#)
2. [Seleccionar, filtrar y resumir con SQL](#)
3. [JOIN y validación de cardinalidad](#)

4. CTE, ventanas, fechas y nulos
5. MongoDB y documentos
6. DynamoDB y patrones de acceso
7. Warehouse, lakehouse y consultas asistidas

Práctica

Resuelve [la consulta de conversión](#) y compara con [la solución](#).

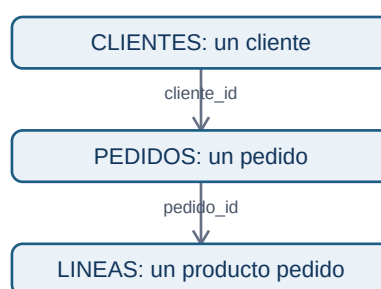
Modelo relacional, tablas y grano

Objetivos y prerequisites

Comprenderás una base de datos relacional como un conjunto de tablas conectadas, antes de escribir una consulta.

Una base de datos guarda información para que programas y personas puedan consultarla de forma consistente. En el modelo **relacional**, una tabla representa una entidad o hecho: clientes, pedidos o eventos. Una fila es una observación y una clave identifica o conecta filas.

La pregunta principal antes de SQL es el **grano**: ¿una fila de pedidos representa un pedido completo o una línea de producto? Sumar importes después de unir tablas sin responderla puede duplicar ingresos.



El diagrama muestra relaciones uno a muchos: un cliente puede tener pedidos y un pedido varias líneas. La clave no es una decoración: define qué combinaciones son válidas.

Resumen

SQL opera sobre tablas, pero el análisis depende de grano y claves. Sigue con [selección, filtro y agregación](#).

Seleccionar, filtrar y resumir con SQL

Objetivos y prerequisites

Escribirás una consulta legible que responda una pregunta concreta y comprobarás qué filas excluye.

SQL es un lenguaje declarativo: describes qué resultado quieres, no los pasos internos exactos. Para contar pedidos pagados por canal:

```
SELECT canal, COUNT(*) AS pedidos
FROM pedidos
WHERE estado = 'pagado'
GROUP BY canal
ORDER BY pedidos DESC;
```

WHERE filtra filas antes de agrupar; GROUP BY define el nivel del resultado. COUNT(*) cuenta filas, pero COUNT(DISTINCT pedido_id) cuenta pedidos únicos. Elegir uno es una definición de métrica, no una preferencia sintáctica.

Error habitual

Filtrar un periodo con texto o sin zona horaria explícita puede incluir o excluir registros inesperados. Inspecciona datos de borde y declara la ventana temporal.

Resumen

Una consulta profesional hace visibles medida, población y grano. Continúa con [JOIN y cardinalidad](#).

JOIN y validación de cardinalidad

Objetivos y prerequisites

Combinarás tablas sin multiplicar accidentalmente registros.

Un JOIN une filas por una clave. INNER JOIN conserva coincidencias; LEFT JOIN conserva todas las filas izquierdas y muestra ausencia de coincidencia. Antes de unir, declara cardinalidad: uno a uno, uno a muchos o muchos a muchos.

```
SELECT p.pedido_id, c.pais
FROM pedidos p
LEFT JOIN clientes c ON p.cliente_id = c.cliente_id;
```

Si `clientes` contiene dos filas para el mismo `cliente_id`, cada pedido aparecerá dos veces. Comprueba recuento antes y después, claves duplicadas y nulos introducidos por la unión. Un SQL que ejecuta no prueba que el resultado sea válido.

Resumen

La cardinalidad es un supuesto analítico que debes validar. Sigue con [SQL analítico](#).

CTE, ventanas, fechas y nulos

Objetivos y prerequisites

Escribirás consultas por pasos y compararás una fila con su contexto sin destruir el detalle.

Una CTE (`WITH`) da nombre a un resultado intermedio y mejora la revisión. Las funciones de ventana calculan sobre un grupo sin reducirlo: `ROW_NUMBER()` ordena pedidos por cliente, `SUM(...)` `OVER(...)` construye acumulados y `LAG()` compara con el valor anterior.

Las fechas necesitan una zona y una granularidad; los nulos no equivalen automáticamente a cero. Usa `COALESCE` solo cuando la regla de negocio diga qué significa la ausencia.

Resumen

La consulta mantenible separa pasos, documenta supuestos y conserva el detalle necesario para validar.

MongoDB y documentos

Objetivos y prerequisites

Entenderás qué problema resuelve un documento flexible y qué preguntas siguen siendo analíticas.

Un documento puede guardar información anidada de una entidad, por ejemplo un pedido con dirección y líneas. MongoDB almacena colecciones de documentos y permite filtros y pipelines de agregación. Esta flexibilidad ayuda cuando la forma de los datos cambia o una aplicación lee el agregado completo.

No significa “sin modelo”. Debes definir identificadores, campos opcionales, versiones y cómo se agregan importes. Documentos duplicados o esquemas inconsistentes complican análisis histórico y comparaciones.

AI asistiendo consultas

Una herramienta que genera un filtro o pipeline desde lenguaje natural produce un borrador. Revisa colección, filtros, periodos, campos sensibles, coste e interpretación del resultado antes de ejecutarlo o compartirlo.

Resumen

NoSQL cambia el modelo de almacenamiento; no elimina definición de métrica ni validación.

DynamoDB y patrones de acceso

Objetivos y prerequisites

Comprenderás por qué algunas bases clave-valor se diseñan empezando por las consultas que una aplicación necesita.

DynamoDB organiza registros alrededor de una clave de partición y, opcionalmente, una clave de ordenación. Está pensado para accesos predecibles a gran escala: “dame los pedidos de este cliente ordenados por fecha”, no para unir libremente cualquier tabla después.

Antes de modelar, enumera patrones de acceso, volumen, frecuencia y orden requerido. Diseñar solo por “entidades bonitas” puede producir consultas caras o imposibles. Para análisis amplio se suele extraer a un warehouse o lakehouse.

Resumen

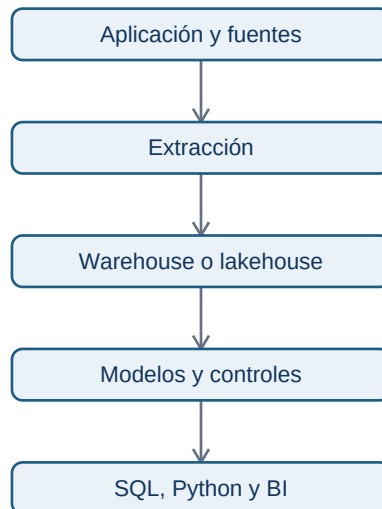
El modelo operativo optimiza accesos conocidos; el modelo analítico optimiza preguntas y historia.

Warehouse, lakehouse y consultas asistidas

Objetivos y prerequisites

Relacionarás fuentes operacionales con el entorno donde se preparan datos para análisis.

Un sistema operacional registra transacciones para que la aplicación funcione. Un **warehouse** organiza datos históricos y modelados para consulta analítica; un **lakehouse** combina almacenamiento flexible con capacidades analíticas. La arquitectura suele extraer, transformar y documentar datos antes de dashboards, SQL o Python.



La AI puede acelerar un borrador de consulta, pero no conoce por defecto el grano, la semántica, permisos o coste. Contrasta siempre resultado, plan de consulta y definición de métrica.

Resuelve la [consulta de conversión](#) antes de consultar la solución.

Bloque 10 - Métricas, KPIs y analítica de producto

Propósito del bloque

Este bloque enseña a construir un sistema de medición para un producto o negocio tecnológico. No se trata de aprender una lista de siglas ni de abrir un dashboard: se trata de decidir qué representa valor, cómo medirlo de forma consistente, qué señales deben activar una investigación y cómo evitar que una métrica optimizada localmente perjudique al producto.

Resultado de salida

Al terminar podrás tomar una pregunta como “queremos crecer” y convertirla en un árbol de métricas con definiciones, instrumentación, guardrails, segmentos y una decisión asociada. También sabrás revisar un funnel, una cohorte o un dashboard de Amplitude sin aceptar sus cifras ciegamente.

Prerrequisitos

- Bloque 00: preguntas y decisiones.
- Bloques 05–08: datos tabulares, exploración y estadística básica.
- Bloque 09: comprensión básica de fuentes y consultas.

Lecciones

1. Dato, medida, métrica, indicador y KPI.
2. Contrato de una métrica: definición que otra persona puede repetir.
3. Objetivos, North Star, árboles y guardrails.
4. Baselines, objetivos, benchmarks, ratios y comparaciones.
5. Funnels: definición, instrumentación y diagnóstico de conversión.
6. Cohortes, retención, churn y segmentación.
7. Adquisición, engagement, monetización y métricas de valor.
8. Experimentación, Goodhart y decisiones bajo incertidumbre.
9. Catálogo de métricas, tracking plan y Amplitude.

Práctica

Cuando termines las tres primeras lecciones, realiza [el ejercicio de árbol de métricas](#). No abras la [solución](#) hasta haber definido tus propios supuestos.

10.1 Dato, medida, métrica, indicador y KPI

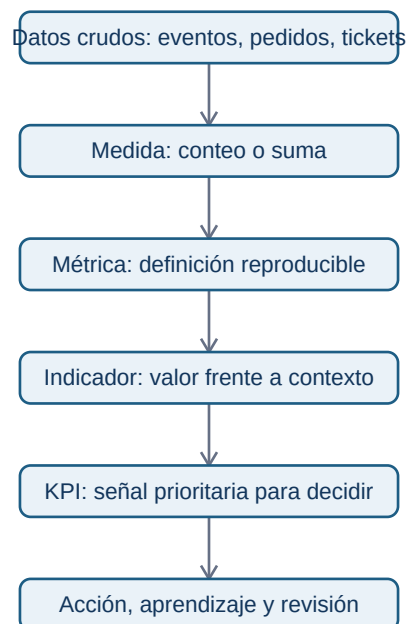
Objetivos

Al terminar esta lección podrás diferenciar los cinco términos que más se confunden en una conversación de negocio y detectar por qué una frase como “la métrica ha subido” puede ser inútil si no está definida.

El problema no es contar; es representar una realidad

Una empresa tecnológica produce muchas huellas: eventos de aplicación, pedidos, tickets de soporte, pagos, campañas y cambios de código. Ninguno de esos registros, por sí solo, responde una pregunta de negocio. El trabajo del analista consiste en convertirlos en una representación explícita y limitada de una realidad: quién hizo qué, cuándo, bajo qué condiciones y por qué nos importa.

Un **dato** es un valor registrado: `usuario_id=42`, `evento="checkout_completed"`, `importe=39.90`. Una **medida** es una operación elemental sobre datos, como contar eventos o sumar importes. Una **métrica** añade una definición reutilizable y un propósito: por ejemplo, “usuarios activos semanales”, calculados como usuarios únicos que realizan una acción de valor entre lunes y domingo. Un **indicador** interpreta una métrica respecto a un contexto: “la activación está 2 puntos por debajo del objetivo”. Un **KPI** es el indicador elegido para gobernar una prioridad importante y al que se asigna responsabilidad y seguimiento.



La flecha no significa que toda medida acabe siendo un KPI. La mayoría no debería serlo. Si una organización convierte cada número visible en un KPI, nadie sabe qué priorizar y se optimizan cifras irrelevantes.

Ejemplo: “usuarios activos” no es una métrica hasta que la definas

Supón que tres equipos presentan el mismo dashboard. Producto llama activo a quien abre la aplicación; Marketing llama activo a quien recibe un correo; Finanzas llama activo a quien paga. Los tres pueden estar usando datos correctos y aun así discutir sobre cifras incompatibles. El problema no

es una fórmula: es una definición incompleta.

Una definición mínima de “usuario activo semanal” podría ser: “usuario identificado que completa al menos una acción de valor entre las 00:00 del lunes y las 23:59 del domingo, en la zona horaria del producto; se excluyen empleados, cuentas de prueba y eventos enviados por sistemas automáticos”. Ahora se puede calcular, discutir y cambiar de forma controlada.

La métrica no debe sustituir la pregunta

Una métrica es buena cuando ayuda a tomar una decisión. “Número de clics” rara vez es una decisión completa. “Porcentaje de usuarios nuevos que completa el primer proyecto en 7 días, segmentado por plataforma” puede orientar si hay que revisar onboarding, compatibilidad móvil o adquisición.

Antes de aceptar una métrica, plantea estas preguntas:

1. ¿Qué comportamiento o resultado pretende representar?
2. ¿Quién entra en la población y quién no?
3. ¿Qué evento o fuente se considera evidencia?
4. ¿Qué ventana temporal y zona horaria aplican?
5. ¿Qué decisión cambiaría si la métrica mejora o empeora?

Si la quinta pregunta no tiene respuesta, probablemente estás ante una cifra decorativa o exploratoria, no ante un KPI.

Errores frecuentes

- Llamar KPI a todo lo que aparece en un dashboard.
- Confundir volumen con valor: más registros no implica más clientes satisfechos.
- Comparar métricas con definiciones o periodos distintos.
- Usar una media global cuando segmentos diferentes tienen comportamientos opuestos.
- Olvidar que un dato puede ser correcto técnicamente y engañoso para la decisión.

Comprobación

Clasifica estas frases: “importe de una transacción”, “ingresos mensuales por cliente activo”, “la retención está por debajo del mínimo aceptable”, “retención a 30 días es un KPI del objetivo de sostenibilidad”. Después explica qué información falta en cada una para que sea reproducible.

10.2 Contrato de una métrica: una definición que otra persona puede repetir

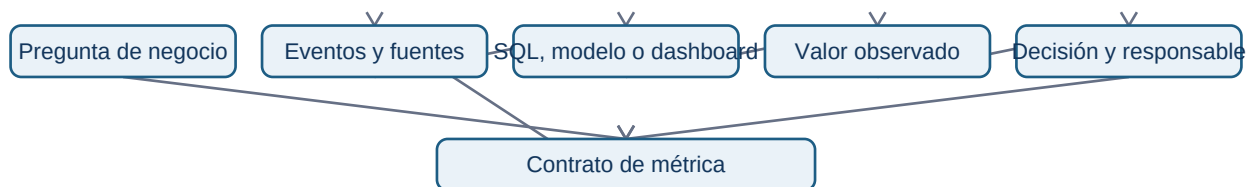
Objetivos

Aprender a especificar una métrica como un contrato: una pieza de documentación y lógica que permite que producto, datos, finanzas y dirección hablen del mismo número.

Por qué una fórmula no basta

Escribir $\text{conversion} = \text{compras} / \text{visitas}$ parece claro hasta que aparecen preguntas reales: ¿visitas de quién? ¿una visita por sesión, dispositivo o usuario? ¿la compra debe ocurrir el mismo día? ¿se cuentan devoluciones? ¿qué ocurre si el tracking se duplicó durante dos horas? La fórmula es solo una parte de la definición.

Un contrato de métrica elimina ambigüedad antes de que el dashboard llegue a una reunión. Debe ser breve, pero suficiente para que otra persona pueda reproducirla sin interpretar intenciones.



El último retorno importa: una definición no es eterna. Si cambia el producto, el comportamiento de valor o la fuente, se revisa el contrato y se documenta el cambio. No se sobrescribe silenciosamente la historia.

Los siete campos mínimos

1. **Nombre y propósito.** “Activación a 7 días” y la decisión que pretende informar.
2. **Fórmula.** Numerador, denominador, unidades y tratamiento de cero.
3. **Población.** Quién es elegible, exclusiones y regla de identidad.
4. **Grano.** Usuario, cuenta, pedido, sesión, evento o día.
5. **Ventana temporal.** Inicio, fin, zona horaria y posible retraso de datos.
6. **Fuentes y lógica.** Eventos, tablas, filtros, versión de modelo y reglas de calidad.
7. **Propietario y límites.** Quién responde por la definición y qué no representa la métrica.

Ejemplo completo: activación de una aplicación B2B

Propósito: saber si usuarios nuevos alcanzan el primer resultado de valor durante su primera semana y decidir qué paso de onboarding debe mejorarse.

Fórmula: usuarios únicos que crean un proyecto y ejecutan su primera consulta dentro de los siete días siguientes al registro / usuarios nuevos elegibles registrados en el mismo periodo. La métrica se expresa como porcentaje.

Población: usuarios humanos con cuenta verificada; se excluyen empleados, sandboxes internas, bots y migraciones masivas. El identificador estable es `account_user_id`.

Grano y ventana: cada usuario aporta una vez a su cohorte de registro. El día cero es el día de registro en UTC. Se esperan siete días completos antes de cerrar una cohorte.

Fuente: tabla de usuarios para registro, eventos `project_created` y `query_executed` para el criterio de valor. Validaciones: no más de un 1 % de eventos sin identificador y reconciliación diaria con logs de backend.

Límites: medir activación no demuestra retención ni satisfacción. Un usuario puede completar la acción por curiosidad y no volver; por eso se acompaña de retención y métricas de calidad.

Versionado y cambios

Si el producto cambia y ahora una integración automática crea proyectos por el usuario, la definición anterior deja de medir la misma conducta. Mantener la misma etiqueta sin documentarlo rompe comparaciones históricas. Decide entre conservar la versión antigua, crear una v2 o recalcular el histórico si existe una regla equivalente. La elección debe quedar registrada.

Comprobación

Escribe el contrato de “tasa de conversión de prueba a pago”. Incluye una exclusión razonable, una decisión que informaría y una limitación que impediría interpretarla como salud total del producto.

10.3 Objetivos, North Star, árboles y guardrails

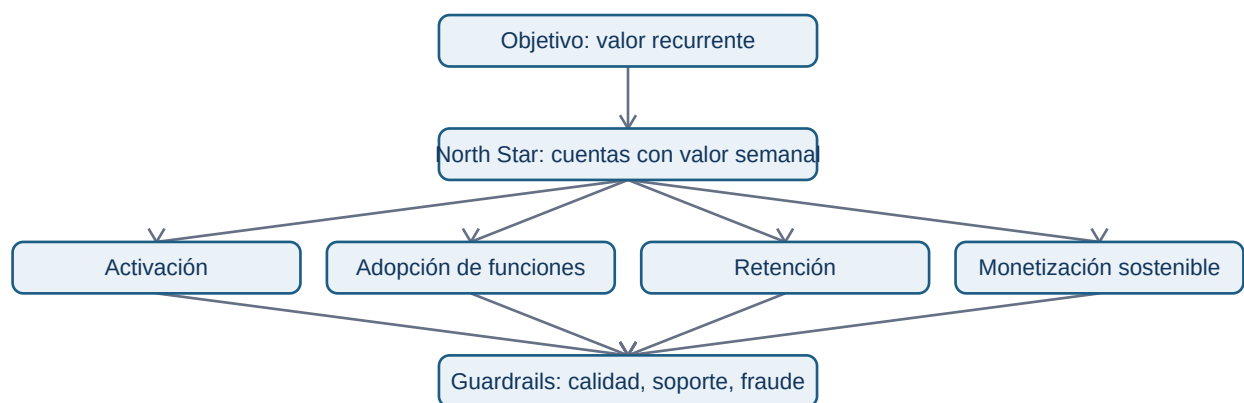
Objetivos

Relacionar la estrategia de un producto con métricas operables sin caer en la trampa de gestionar una organización con una única cifra.

De estrategia a sistema de medida

Un objetivo formula una dirección: “hacer que equipos pequeños obtengan valor recurrente del producto”. Una North Star Metric intenta resumir la entrega de valor de ese objetivo. No sustituye a la estrategia ni resume todas las obligaciones de la empresa; sirve como punto de coordinación.

Una North Star útil debe estar relacionada con valor para el cliente, ser medible con suficiente calidad, responder a acciones de equipos y no ser tan fácil de manipular que incentive un comportamiento dañino. “Usuarios registrados” suele ser demasiado superficial; “cuentas que completan un flujo de valor semanal” suele estar más cerca de la experiencia real, aunque exige una definición cuidadosa.



El árbol no es una cadena causal demostrada automáticamente. Es una hipótesis de negocio: debe contrastarse con análisis, experiencia de producto y experimentos. Su valor está en obligar a explicitar cómo se espera que una acción local contribuya al resultado global.

Métricas de entrada y de resultado

Las métricas de resultado miran el efecto final: ingresos, retención o valor entregado. Son importantes, pero tardan en cambiar. Las métricas de entrada representan comportamientos o condiciones que un equipo puede influir antes: completar onboarding, tiempo hasta primer valor, cobertura de documentación o tasa de errores.

No elijas una métrica de entrada porque sea fácil de mover. El vínculo con el resultado debe ser plausible y medible. Por ejemplo, aumentar notificaciones enviadas puede mejorar una métrica de actividad a corto plazo y empeorar retención por fatiga.

Guardrails: progreso sin daño oculto

Un guardrail es una métrica que limita una optimización. Si el objetivo es elevar conversión, guardrails habituales son tasa de devoluciones, tickets de soporte, latencia, fraude, cancelación o satisfacción. No son métricas secundarias: definen qué tipo de éxito es aceptable.

El fenómeno de Goodhart resume el riesgo: cuando una medida se convierte en objetivo, las personas encuentran maneras de mejorarla sin mejorar lo que pretendía representar. Un equipo puede impulsar activación añadiendo un paso obligatorio que dispara el evento de valor, aunque el usuario no haya recibido valor alguno. El árbol de métricas y los guardrails ayudan a detectar esta distorsión.

Ejemplo de decisión

Un equipo observa menor activación en móvil. Su árbol sugiere revisar el tiempo hasta primer proyecto y el abandono en el permiso de notificaciones. Antes de rediseñar, segmenta por versión, dispositivo y canal; comprueba instrumentación; estima el tamaño de la caída; y decide si necesita un experimento o una corrección técnica. El árbol orienta la investigación, no reemplaza el análisis.

Comprobación

Para una plataforma de cursos, propone una North Star, tres entradas y dos guardrails. Después describe una forma de manipular la North Star sin generar aprendizaje real y cómo lo detectaría un guardrail.

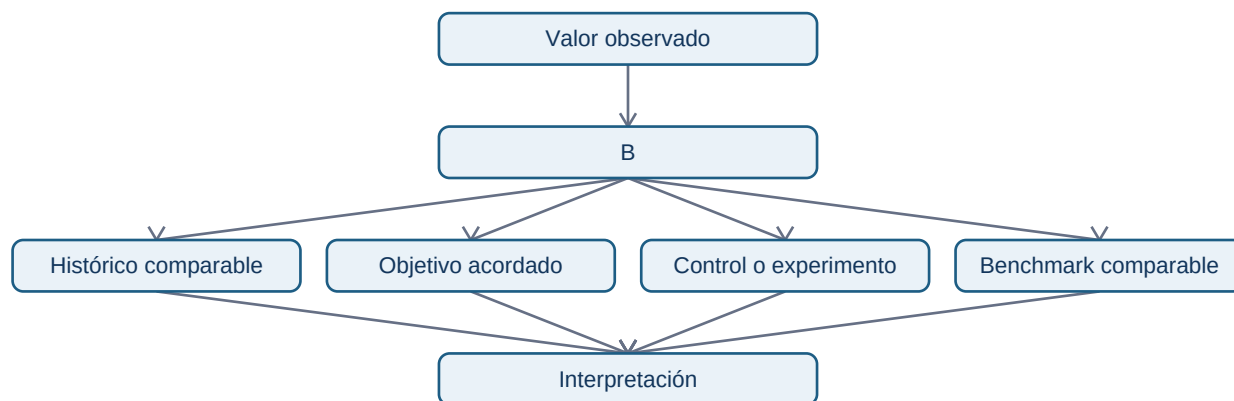
10.4 Baselines, objetivos, benchmarks, ratios y comparaciones

Objetivos

Evitar conclusiones engañosas al comparar una métrica con un periodo, una población o una referencia inadecuados.

Un número aislado casi nunca informa

Decir “la conversión es 4 %” no permite decidir. ¿Es 4 % frente a un objetivo de 3 % o de 8 %? ¿Sube respecto a la semana anterior? ¿La semana anterior tenía una campaña, una caída de tracking o un festivo? Toda métrica necesita una referencia: baseline histórico, objetivo acordado, benchmark externo comparable o grupo de control.



Un benchmark externo puede orientar, pero rara vez es una meta automática: modelo de negocio, mercado, madurez, definición y población pueden diferir. Es más riguroso usarlo para plantear preguntas que para declarar éxito o fracaso.

Absoluto, relativo y normalizado

Una caída de 200 conversiones puede ser enorme para un producto pequeño y trivial para otro. Por eso se combinan recuentos absolutos, tasas, cambios porcentuales y, cuando procede, normalización por población o exposición. La tasa de conversión necesita denominador; los ingresos por usuario necesitan periodo y población; la disponibilidad necesita duración observada.

Evita comparar porcentajes cuando los denominadores son minúsculos. Pasar de 1 a 2 compras es un aumento del 100 %, pero no justifica el mismo lenguaje que pasar de 10 000 a 20 000. Comunica ambos valores.

Segmentos y paradojas

La media global puede mejorar mientras todos los segmentos relevantes empeoran, si cambió la composición de la población. Divide por canal, plataforma, cohorte, región o plan cuando exista una razón de negocio. Pero no busques segmentos hasta encontrar uno “significativo”: define previamente cuáles son plausibles y documenta exploraciones adicionales.

Comprobación

Una conversión mensual sube del 3 % al 4 %. Escribe cinco datos que pedirías antes de celebrar la mejora y una manera de comunicarla sin exageración.

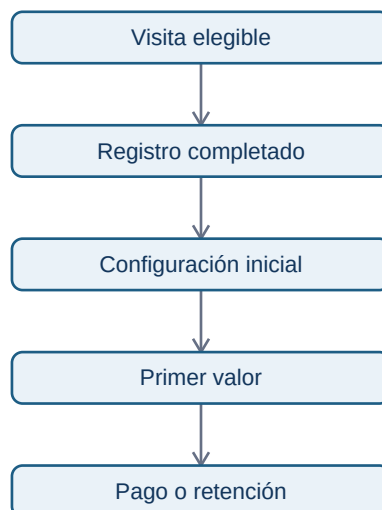
10.5 Funnels: definición, instrumentación y diagnóstico de conversión

Objetivos

Construir un funnel que represente un recorrido real del usuario y diagnosticar pérdidas sin confundir eventos técnicos con progreso de valor.

Qué mide un funnel

Un funnel compara cuántas entidades pasan por una secuencia de pasos definidos. La entidad puede ser usuario, cuenta, pedido o sesión; escogerla cambia la respuesta. Un funnel de onboarding por usuario y un funnel de checkout por pedido no son intercambiables.



Cada flecha es una hipótesis sobre un recorrido. Define orden, ventana máxima, repetición de eventos, exclusiones y tratamiento de usuarios que entran a mitad del proceso. Si una persona completa pasos en varios dispositivos, necesitas una regla de identidad antes de calcular.

Pérdida no significa causa

Que el mayor abandono ocurra entre B y C no demuestra que el formulario sea el problema. Puede haber tráfico de baja intención, una incompatibilidad de navegador, un cambio de precio o un evento que no se está registrando. El funnel localiza dónde investigar; logs, sesiones, segmentación,

cualitativo o experimentos ayudan a explicar por qué.

Instrumentación mínima

Para cada paso documenta nombre, condición de éxito, propiedades, cuándo se envía el evento y qué sistemas pueden generarlo. Distingue “botón pulsado” de “acción completada en backend”. El primer evento mide intención; el segundo confirma resultado. Ambos pueden ser útiles, pero responden preguntas distintas.

Ejemplo de diagnóstico

Si la conversión cae solo en Android tras una versión, compara el funnel por versión de app, modelo de dispositivo y error técnico. Si el evento de “registro completado” cae pero las cuentas existen en base de datos, el problema puede ser instrumentación. La investigación responsable informa de ambas posibilidades antes de atribuir culpa al producto.

Comprobación

Define un funnel de compra de suscripción. Indica entidad, pasos, ventana y un evento técnico que no usarías como criterio final de conversión.

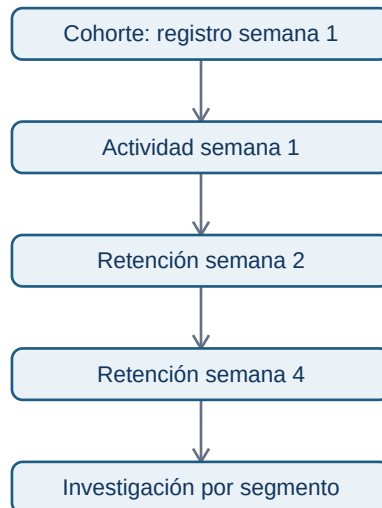
10.6 Cohortes, retención, churn y segmentación

Objetivos

Interpretar cuándo una población vuelve, se mantiene o abandona, y evitar comparar cohortes que no son equivalentes.

La cohorte da contexto temporal

Una cohorte agrupa entidades que comparten una condición de entrada: por ejemplo, usuarios registrados en la misma semana o cuentas que activaron una funcionalidad. La retención pregunta qué proporción vuelve a realizar una acción definida después de esa entrada. Sin cohorte, una media de usuarios activos mezcla generaciones de producto, campañas y antigüedad.



Define con precisión evento de entrada, evento de retorno, intervalo y tipo de retención. La retención clásica exige volver en un periodo concreto; la no acotada permite volver en o después de cierto día. Ambas son válidas si se nombran correctamente.

Churn no es simplemente “no activo”

Churn puede referirse a cancelación contractual, inactividad durante un umbral o pérdida de ingresos. Un SaaS anual y una app gratuita no usan la misma definición. Declara el horizonte y la población: una cuenta que aún no tuvo oportunidad razonable de renovar no debe entrar en una tasa de cancelación.

Segmentación con propósito

Segmenta cuando exista una hipótesis operativa: canales que traen usuarios de distinto valor, planes con onboarding distinto, países con diferencias regulatorias o cuentas con distintos tamaños. No uses segmentos como excusa para ocultar la métrica global; combina ambos niveles y declara denominadores.

Comprobación

Compara dos cohortes con retención día 30 del 20 % y 25 %. Enumera información necesaria antes de afirmar que la segunda experiencia de onboarding es mejor.

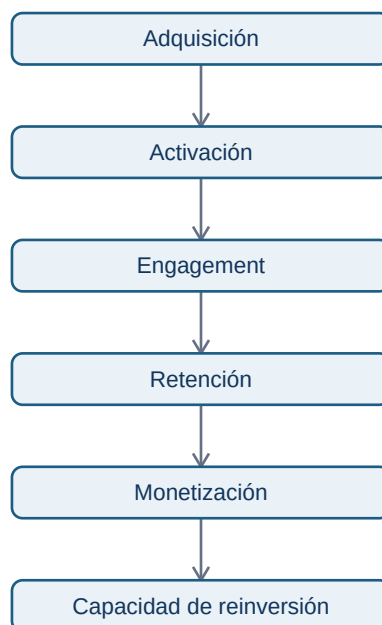
10.7 Adquisición, engagement, monetización y valor

Objetivos

Relacionar métricas de distintas etapas del producto sin convertirlas en una colección desconectada de siglas.

El recorrido económico y de valor

Adquisición responde quién llega; activación responde quién obtiene primer valor; engagement responde con qué frecuencia y profundidad se usa; retención responde quién vuelve; monetización responde qué valor económico sostiene el servicio. No existe una secuencia universal, pero dibujar la relación evita optimizar una etapa contra otra.



DAU, WAU y MAU son recuentos de actividad; el cociente DAU/MAU se usa a veces como aproximación a frecuencia, pero solo es interpretable con una definición de actividad estable. ARPU, CAC y LTV ayudan a hablar de economía unitaria, pero dependen de costes, horizontes y atribución. No los presentes como verdades universales.

La métrica de valor más útil no siempre es ingreso. En un producto B2B puede ser informes entregados; en una plataforma educativa, actividades significativas completadas; en una infraestructura, tareas ejecutadas con éxito. El valor debe conectar una necesidad del usuario con una viabilidad de negocio.

Comprobación

Elige un producto y propone una métrica de valor, una de engagement, una de monetización y una advertencia sobre la relación entre ellas.

10.8 Experimentación, Goodhart y decisiones bajo incertidumbre

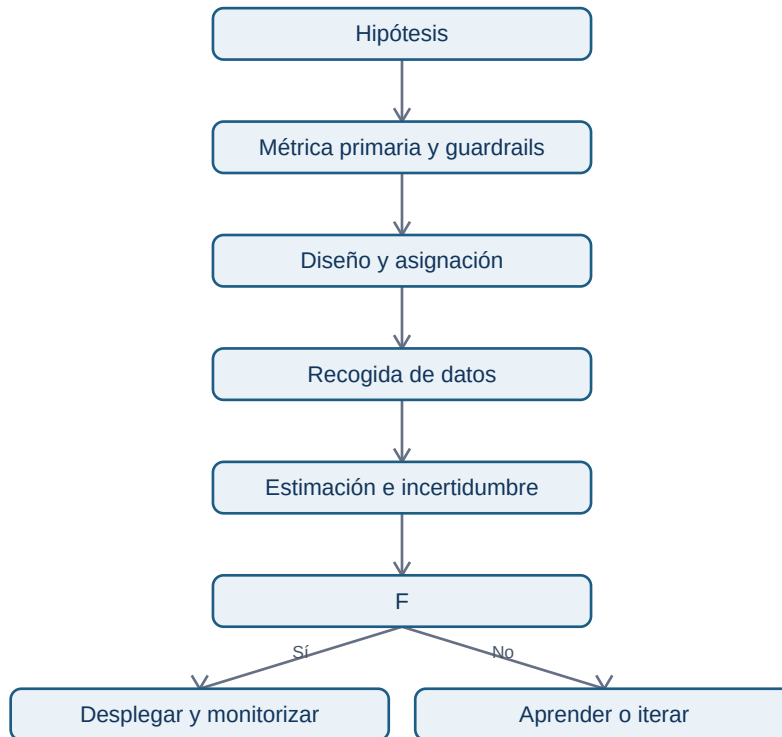
Objetivos

Usar métricas para evaluar cambios sin convertir una mejora puntual en una certeza ni incentivar comportamientos que dañen el producto.

Antes del experimento

Define hipótesis, población, métrica primaria, guardrails, duración y criterio de decisión antes de mirar los resultados. Si la métrica cambia después de conocer los datos, aumentas la probabilidad de encontrar una historia convincente pero falsa.

La estadística ayuda a medir incertidumbre; no decide por sí sola. Una diferencia puede ser compatible con ruido, demasiado pequeña para justificar coste o perjudicial en un segmento. Comunica magnitud absoluta, relativa, intervalo, tamaño de muestra, duración y límites.



Goodhart en la práctica

Una medida se degrada cuando las personas optimizan el indicador en vez del fenómeno. Ejemplos: forzar clics para elevar engagement, retrasar una cancelación para reducir churn del mes, o dividir un ticket para mejorar tiempo de primera respuesta. Los guardrails, auditorías cualitativas y métricas complementarias no eliminan el riesgo, pero lo hacen visible.

Comprobación

Propón un experimento para elevar activación. Incluye métrica primaria, dos guardrails, decisión de parada y una posible forma de Goodhart.

10.9 Catálogo de métricas, tracking plan y Amplitude

Objetivos

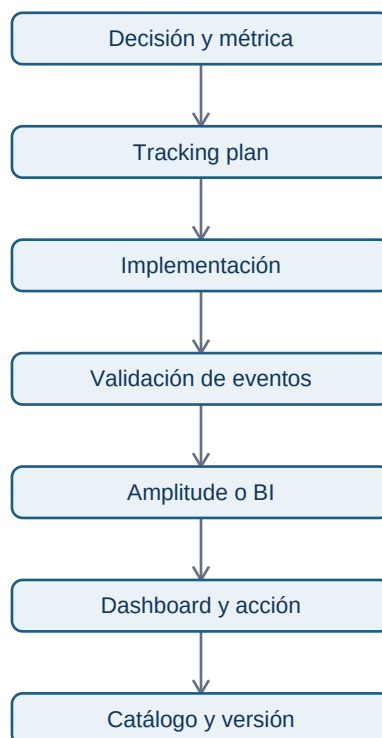
Comprender que la confianza en un dashboard depende de un sistema de gobierno: definiciones, eventos, propiedad, cambios y calidad.

Catálogo de métricas

Un catálogo es el lugar donde se encuentran nombre, propósito, definición, fórmula, fuentes, propietario, versiones, dashboards y consumidores de una métrica. Evita que cada equipo reconstruya “usuarios activos” desde cero y permite investigar diferencias de forma trazable.

Tracking plan

Antes de instrumentar, documenta qué eventos representan interacciones importantes, qué propiedades permiten segmentar y cómo se identifica a usuario, cuenta o dispositivo. Define también eventos prohibidos: no envíes PII innecesaria a herramientas de analítica. El plan debe incluir validación de cobertura y un proceso de cambio cuando el producto evolucione.



Amplitude como ejemplo, no como sustituto del criterio

Amplitude permite trabajar con eventos, propiedades, funnels, cohorts, retención y dashboards. Eso no le concede autoridad sobre la definición: una visualización correcta sobre eventos mal instrumentados

sigue siendo engañosa. Valida primero identidad, latencia, duplicados, eventos de servidor frente a cliente y cambios de versión.

Una práctica sana consiste en revisar cada métrica con tres capas: definición de negocio, lógica técnica y comportamiento observado. Si las tres no coinciden, el trabajo no está terminado.

Comprobación

Escribe tres eventos y dos propiedades para medir activación de un producto. Indica qué dato no recogerías por privacidad y cómo comprobarías que el evento llega correctamente.

Bloque 11 - Series temporales

Propósito

Analizar pedidos diarios de una aplicación, distinguir patrones temporales de cambios reales y construir previsiones útiles para planificar capacidad e inventario. El objetivo no es memorizar modelos: es definir, validar y comunicar una predicción que otra persona pueda cuestionar.

Caso continuo: pedidos diarios de Lumen

Lumen es una aplicación de comercio local. Operaciones debe decidir cada lunes cuántos repartidores reservar para los 14 días siguientes. La métrica es `pedidos_completados_diarios`; cada observación es un día en la zona horaria de Madrid. El bloque parte de este caso y conserva su contrato, datos disponibles y riesgos a lo largo de todas las lecciones.

Lecciones

1. Contrato de previsión y calidad temporal
2. Tendencia, estacionalidad, calendario y rupturas
3. Lags, autocorrelación y ventanas móviles
4. Baselines y un modelo sencillo
5. Validación walk-forward y fuga de futuro
6. Métricas de previsión y coste de error
7. Intervalos de predicción y calibración
8. Rupturas, monitorización y operación
9. Laboratorio reproducible de demanda

Práctica

Plantea una [previsión de demanda](#), ejecuta el [laboratorio reproducible](#) y compara con la [solución razonada](#).

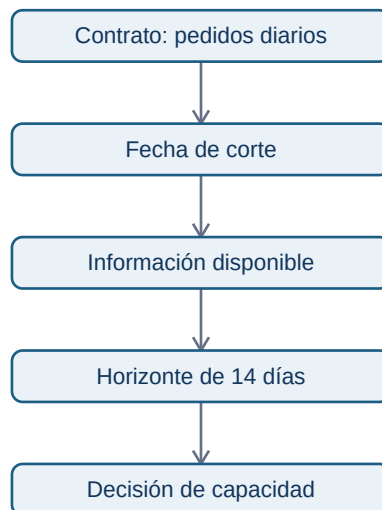
Contrato de previsión y calidad temporal

Objetivos y prerequisites

Definirás qué se predice, cuándo se predice y qué datos son legítimos antes de mirar un gráfico o elegir un modelo.

Una **serie temporal** es una medida observada en momentos ordenados. En Lumen una fila representa un día; la variable objetivo es el número de pedidos_completados; la frecuencia es diaria; la zona es Europe/Madrid; el horizonte son los 14 días siguientes; y la fecha de corte es el domingo a las 23:59. Operaciones decide cuántos repartidores reservar el lunes usando solo información anterior al corte.

Esto responde a la pregunta “¿qué contrato evita que una previsión sea una cifra sin contexto?”



El diagrama es una secuencia de decisión: no se puede usar una campaña conocida el miércoles para predecir el lunes anterior. El contrato hace visible el momento en que el dato se vuelve utilizable.

Antes de modelar, construye un calendario completo. Un día sin fila puede significar cero pedidos, una caída del sistema de captura o una fuente incompleta; son tres hechos distintos. Comprueba duplicados, zona horaria, horas de cambio estacional, cobertura, agregación y cambios de definición. Si desde julio “pedido completado” excluye pedidos parcialmente reembolsados, no compares niveles

sin documentar la ruptura.

Ejemplo mínimo: si el 6 de enero no aparece en el archivo, no rellenas automáticamente con cero. Primero contrastas el registro operacional; solo después decides si es un cero real, un ausente o un día que debe excluirse.

Resumen

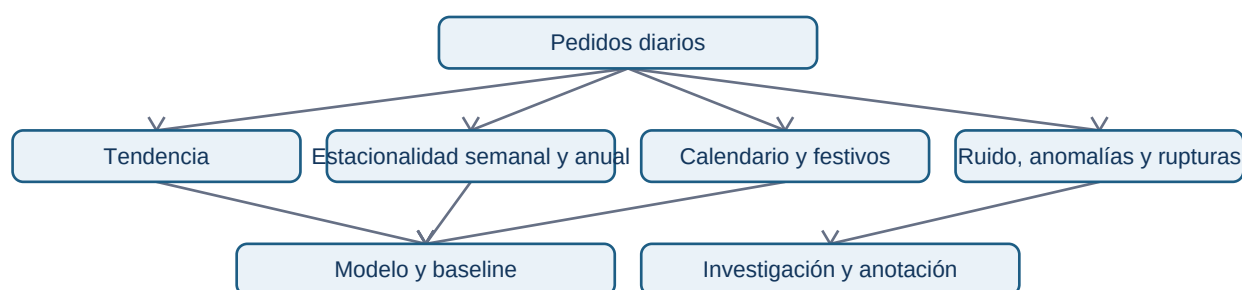
Una serie fiable empieza por un contrato, un calendario y una métrica estable. Continúa con **tendencia**, **estacionalidad** y **rupturas**.

Tendencia, estacionalidad, calendario y rupturas

Objetivos y prerequisites

Separarás patrones sostenidos, repeticiones de calendario, ruido y cambios estructurales antes de atribuir una causa.

Una serie puede contener **tendencia** (movimiento de largo plazo), **estacionalidad** (patrón que se repite por día, semana o año), ciclos, ruido y rupturas. En Lumen los viernes pueden superar a los martes; noviembre puede contener un pico de campañas; y un cierre de zonas de reparto puede producir un cambio de nivel. Una caída de lunes a domingo puede ser normal; una caída frente al mismo lunes de semanas comparables merece investigación.



Los componentes son ramas paralelas: no ocurren uno después de otro. Una descomposición puede ser **aditiva** si los efectos se suman aproximadamente (por ejemplo, +20 pedidos cada viernes) o **multiplicativa** si la amplitud crece con el nivel (por ejemplo, un 20 % más). Una transformación logarítmica puede ayudar en el segundo caso, pero no es una obligación ni admite ceros sin tratamiento explícito.

Descomponer no prueba causas. Un cambio de nivel puede coincidir con una campaña, una incidencia, un festivo, falta de stock o error de medición. Cruza eventos y segmentos antes de explicarlo; registra la ruptura para no entrenar un modelo que la interprete como estacionalidad permanente.

Resumen

Compara contra referencias temporales adecuadas, no solo contra el periodo anterior. Sigue con [lags](#), [autocorrelación](#) y [ventanas móviles](#).

Lags, autocorrelación y ventanas móviles

Objetivos y prerequisites

Usarás valores pasados para describir dependencia temporal sin introducir información futura.

Un **lag** es un valor retrasado: `pedidos_t-1` es ayer y `pedidos_t-7` es el mismo día de la semana anterior. La **autocorrelación** resume cuánto se parece la serie a sí misma tras uno o varios retrasos. Una ACF alta en el lag 7 sugiere patrón semanal, no causalidad ni permiso para copiar siete días sin evaluar.

Una media móvil de 7 días suaviza ruido al promediar solo observaciones anteriores. Para predecir el 10 de marzo, su ventana puede usar del 3 al 9, nunca del 11 al 16. La regla evita una fuga de futuro muy común cuando se calculan ventanas sobre toda la tabla.

Ejemplo: si Lumen tuvo 102 pedidos ayer, 118 hace una semana y una media móvil de 110, esos tres números pueden alimentar modelos distintos. Cada variable contiene una hipótesis: continuidad inmediata, patrón semanal o nivel suavizado.

El horizonte importa: prever mañana y prever seis meses son problemas distintos. Declara qué información estaba disponible en el momento de hacer cada previsión; usar datos futuros por accidente produce resultados irreales. En la siguiente lección convertirás estas referencias en baselines comparables.

Resumen

Lags y ventanas describen dependencia; no prueban que una acción cause pedidos. Sigue con [baselines](#) y [un modelo sencillo](#).

Baselines y un modelo sencillo

Objetivos y prerequisites

Compararás referencias honestas antes de elegir un modelo más elaborado.

Un baseline responde “¿qué lograríamos sin sofisticación?”. Para Lumen compara: naïve (repetir ayer), seasonal naïve (repetir el mismo día de la semana anterior) y media móvil de siete días. Son modelos explícitos, reproducibles y difíciles de superar cuando hay fuerte patrón semanal.

Como modelo sencillo adicional, una regresión con tendencia y variables de día de semana puede estimar nivel y estacionalidad. No es automáticamente mejor: solo se conserva si mejora de forma estable al baseline y su coste/interpretación compensa. Métodos como ETS, ARIMA o modelos de aprendizaje automático se estudian después de dominar esta comparación.

Ejemplo: si la media móvil gana en semanas tranquilas pero pierde sistemáticamente los viernes, la referencia estacional puede ser preferible. No elijas por una única semana ni por una gráfica atractiva.

El siguiente paso es evaluar respetando la flecha del tiempo. Continúa con [validación walk-forward y fuga de futuro](#).

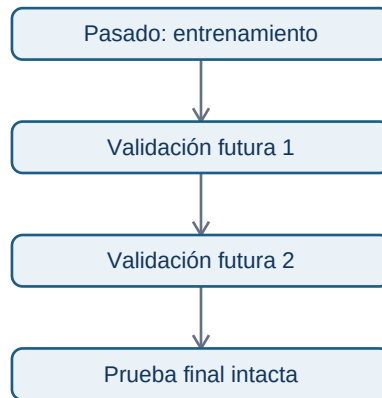
Plantea la [previsión de demanda](#). En modelos posteriores aprenderás predicción supervisada, pero conservarás esta regla: validación coherente con el momento de decisión.

Validación walk-forward y fuga de futuro

Objetivos y prerequisites

Validarás una previsión como se usaría en producción: entrenar con pasado y comprobar contra futuro todavía desconocido.

Una partición temporal no se baraja. En Lumen puedes entrenar hasta septiembre, validar octubre-noviembre, ajustar una única vez y reservar diciembre como prueba final. Para conocer estabilidad, el enfoque **walk-forward** avanza sucesivos cortes: se predice la semana siguiente, se compara con lo observado y se avanza.



Cada bloque está en orden temporal. La prueba final no decide parámetros ni umbrales; estima cómo habría rendido el proceso al desplegarlo.

Una **fuga de información** ocurre si una variable usa el futuro: una media móvil centrada, normalizar usando todos los meses o incluir un precio que se fijó después de la fecha de corte. Un resultado excepcionalmente bueno merece investigar fuga antes de celebrarlo.

Resumen y práctica

La validación simula el momento de decisión. Sigue con [métricas de previsión](#).

Métricas de previsión y coste de error

Objetivos y prerequisites

Elegirás cómo medir un error de previsión según la decisión, la escala y el coste de equivocarse.

El error de un día es $\text{real} - \text{predicción}$. **MAE** promedia su valor absoluto y se interpreta en la unidad del negocio: “nos equivocamos 12 pedidos al día”. **RMSE** eleva errores al cuadrado antes de promediar y penaliza más fallos grandes; puede convenir si quedarse muy corto de capacidad es especialmente grave.

Las métricas porcentuales requieren cuidado. **MAPE** divide por el valor real: no está definido con ceros y sobre-reacciona ante valores pequeños. **SMAPE** reduce algunos problemas, pero también tiene comportamiento difícil cerca de cero. **MASE** escala el error frente a una previsión naïve y permite comparar series de distinto tamaño, siempre que el baseline sea válido.

Ejemplo: predecir 10 pedidos cuando hubo 0 hace que MAPE falle; MAE sigue diciendo 10 pedidos de error. Si reservar repartidores extra cuesta poco pero faltar capacidad cuesta mucho, una métrica

media no basta: comunica también errores por debajo de la demanda y el coste operativo asociado.

Resumen

No existe “la métrica ganadora” fuera de una decisión. Evalúa varias y explica por qué una domina el criterio de lanzamiento. Continúa con [intervalos y calibración](#).

Intervalos de predicción y calibración

Objetivos y prerequisites

Comunicarás incertidumbre de una previsión y comprobarás si el rango prometido se cumple con la frecuencia esperada.

Una previsión puntual de 120 pedidos no expresa todo lo que sabemos. Un **intervalo de predicción** podría comunicar “entre 95 y 145 pedidos con cobertura nominal del 80 %”, bajo el método y supuestos usados. Debe referirse a observaciones futuras, no solo a incertidumbre sobre una media histórica.

La **calibración** pregunta si los intervalos son honestos: de cien días con intervalos al 80 %, aproximadamente ochenta deberían contener el valor real a largo plazo. Cobertura baja indica rangos demasiado estrechos; cobertura muy alta puede indicar rangos inútilmente amplios. Revisa también si el fallo se concentra en viernes, festivos o campañas.

Para Lumen, operaciones puede planificar tres escenarios: bajo, central y alto. La decisión no es “creer” el número central, sino elegir capacidad compatible con el coste de sobre- y sub-reservar.

Límite

Un intervalo no protege frente a un evento fuera de la historia, como cierre de una ciudad o campaña inédita. Es un rango condicionado a datos y supuestos, no una garantía.

Resumen

Una previsión útil incluye incertidumbre verificable. Sigue con [rupturas y monitorización](#).

Rupturas, monitorización y operación

Objetivos y prerequisites

Identificarás cambios que pueden invalidar patrones históricos y diseñarás una respuesta operativa ante errores de previsión.

Una **ruptura estructural** es un cambio por el que la relación aprendida deja de ser estable: cambio de precio, falta de stock, expansión de zonas, campaña, nueva versión de producto o cambio de tracking. No se corrige borrando el punto “raro”; se registra el evento, se evalúa el impacto y se decide si el modelo necesita reentrenamiento o una regla temporal.

Monitoriza error por horizonte, cobertura de intervalos, datos ausentes, frescura y sesgo por día de semana. Una alerta debe indicar umbral, dueño y acción: “si el MAE de siete días supera 20 pedidos dos semanas, revisar calendario, stock y extracción”. Alertar sin responsable crea ruido, no control.

Resumen

Prever es un proceso operativo: datos, modelo, error, aprendizaje y ajuste. Continúa con el [laboratorio reproducible](#).

Laboratorio reproducible de demanda

Objetivos y prerequisites

Aplicarás el contrato temporal, tres baselines, validación ordenada y métricas de error sobre el caso de Lumen.

El script [11-prevision-demanda.py](#) genera de forma determinista 90 días de pedidos diarios con patrón semanal y una ruptura documentada. Divide pasado y futuro, compara naïve, seasonal naïve y media móvil, y calcula MAE, RMSE, MAPE, sMAPE y MASE.

No trates la salida como una respuesta universal: inspecciona qué baseline gana y explica por qué. Cambia la fecha de corte o la ruptura para observar que una métrica global puede ocultar días críticos. Escribe en tu entrega qué información estaría disponible realmente antes de predecir.

Entregable

Resuelve la [práctica de demanda](#), conserva los resultados esperados y contrasta tu razonamiento con la [solución](#).

Bloque 12 - Modelos predictivos para analistas

Propósito

Usar modelos predictivos para estimar resultados, priorizar casos y apoyar decisiones. El objetivo no es maximizar una métrica aislada: es construir una predicción útil, válida y explicable.

Lecciones

1. Decidir si la predicción aporta valor
2. Datos, variables y fuga de información
3. Baselines y familias de modelos
4. Evaluación y coste de errores
5. Interpretación, sesgo y uso responsable

Práctica

Resuelve [el caso de churn](#).

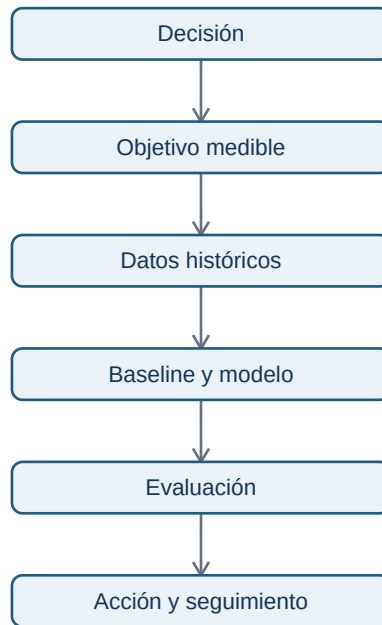
Decidir si la predicción aporta valor

Objetivos y prerequisites

Separarás una pregunta predictiva de una causal y definirás la decisión que una predicción puede mejorar.

Un modelo predictivo usa patrones históricos para estimar un resultado desconocido: probabilidad de abandono, demanda de mañana o importe esperado. Antes de modelar define quién recibirá la predicción, qué acción puede tomar y cuál es el coste de equivocarse.

Predecir riesgo de churn no demuestra por qué alguien abandonará ni qué oferta lo retendrá. Sirve para priorizar contacto; la eficacia de la intervención exige experimento o evidencia causal aparte.



Resumen

Sin decisión y acción, una predicción puede ser interesante pero no valiosa. Sigue con [datos y fuga](#).

Datos, variables y fuga de información

Objetivos y prerequisites

Definirás objetivo, momento de predicción y variables que estarían disponibles entonces.

La **variable objetivo** es lo que se quiere estimar; las variables de entrada describen información disponible antes del resultado. Una **fuga de información** ocurre cuando el modelo usa un dato que solo existe después: una cancelación registrada tras el momento en que querías predecir churn.

Separa entrenamiento, validación y prueba respetando tiempo cuando corresponda. Ajustar transformaciones solo con entrenamiento evita que información del futuro mejore artificialmente la evaluación.

Error habitual

Crear una variable con “número de tickets resueltos” para predecir una baja cuando esos tickets se abren precisamente al iniciar la baja. Un resultado excelente puede ser señal de fuga, no de inteligencia.

Resumen

La pregunta correcta es: “¿qué sabíamos en el instante de decidir?”.

Baselines y familias de modelos

Objetivos y prerequisites

Compararás un modelo con una referencia sencilla antes de usar complejidad adicional.

Un **baseline** puede ser predecir la media, repetir el último valor o clasificar siempre la clase mayoritaria. Si un modelo no lo supera de forma útil, no merece operación ni explicación adicional.

Regresión lineal estima una cantidad; regresión logística estima probabilidad de una clase; árboles capturan divisiones y relaciones no lineales. Ninguna familia es “mejor” sin contexto: más flexibilidad puede sobreajustar y reducir interpretabilidad.

Resumen

Empieza simple y compara con una referencia honesta. Sigue con [evaluación y coste de error](#).

Evaluación y coste de errores

Objetivos y prerequisites

Elegirás métricas según el tipo de resultado y la consecuencia de cada fallo.

Para cantidades, MAE expresa error medio absoluto y RMSE penaliza más fallos grandes. Para clasificación, precisión pregunta cuántos avisos fueron correctos; recall, cuántos casos reales detectaste. No hay métrica universal: en fraude, perder un caso puede costar mucho; en una campaña cara, contactar falsos positivos también.

El umbral de probabilidad transforma un modelo en una acción. Ajustarlo cambia precisión, recall, capacidad operativa y equidad. Evalúa por segmentos relevantes y en datos futuros, no solo una métrica global.

Resumen

La mejor métrica refleja la decisión y sus costes, no el número más alto de una tabla.

Interpretación, sesgo y uso responsable

Objetivos y prerequisites

Comunicarás qué hace un modelo, dónde falla y qué controles necesita antes de automatizar acciones.

Explicar importancia de variables no demuestra causalidad. Una variable puede predecir churn porque está asociada a un canal, no porque modificarla resuelva el abandono. Examina calidad, representatividad, privacidad y posibles proxies de atributos sensibles.

Documenta población, fecha de entrenamiento, objetivo, variables, métricas, umbral, fallos esperados, responsable y monitorización. Si una predicción afecta a personas, conserva revisión humana y mecanismo para detectar daño desigual.

Resuelve el [caso de priorización de churn](#). El siguiente bloque enseña a entregar este trabajo de forma colaborable y reproducible.

Bloque 13 - Herramientas y reproducibilidad

Propósito

Trabajar como analista dentro de un equipo: convertir peticiones en entregables verificables, documentar decisiones y hacer que un análisis pueda repetirse.

Lecciones

1. De petición a ticket analítico
2. Proyecto reproducible y Git
3. Notebooks, scripts y revisiones
4. Instrumentación, tracking plan y Amplitude
5. BI, dashboards y definición de métricas

Práctica

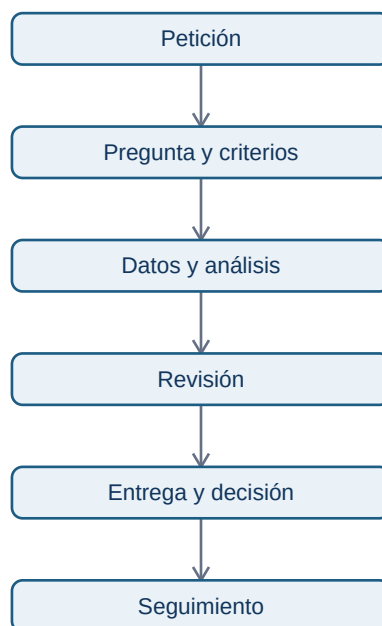
Redacta un [ticket analítico completo](#).

De petición a ticket analítico

Objetivos y prerequisites

Convertirás una petición vaga en un acuerdo de trabajo verificable.

Jira, Linear y herramientas similares registran trabajo y decisiones; no son solo listas. Un ticket analítico debe contener contexto, decisión, pregunta, definición de métrica, alcance, fuentes, criterios de aceptación, responsable y fecha de revisión.



Si alguien pide “analiza el onboarding”, pregunta qué decisión se tomará y qué resultado cambiaría esa decisión. Así evitas producir un dashboard interesante pero inútil.

Práctica

Redacta el [ticket analítico](#) al finalizar el bloque.

Proyecto reproducible y Git

Objetivos y prerequisites

Organizarás un análisis para que otra persona pueda repetir y revisar sus pasos.

Un proyecto separa código, documentación, datos no versionados, resultados derivados y dependencias. Git registra cambios en archivos de texto como consultas, scripts y definiciones; un commit debe explicar una unidad de cambio comprensible.

Reproducibilidad significa poder responder: qué fuente se usó, en qué fecha, qué versión de código, qué parámetros y qué transformaciones generaron el resultado. No subas datos sensibles al repositorio para “hacerlo reproducible”: documenta un acceso seguro y datos sintéticos de ejemplo.

Resumen

Git conserva historia; la documentación conserva significado. Necesitas ambos.

Notebooks, scripts y revisiones

Objetivos y prerequisites

Distinguirás exploración, lógica reutilizable y entrega revisable.

Un notebook mezcla explicación, código y salida: es excelente para aprender, explorar y comunicar. Para lógica repetida, mueve funciones a scripts o módulos probables, con entradas y salidas claras. Ejecuta el notebook de principio a fin antes de compartirlo: celdas ejecutadas fuera de orden producen resultados que nadie puede repetir.

Una revisión de pares busca más que estilo: definiciones, grano, filtros, nulos, uniones, límites y coherencia entre conclusión y evidencia. Deja comentarios que indiquen riesgo y una acción verificable.

Resumen

La herramienta no impone calidad; el flujo de revisión la hace posible.

Instrumentación, tracking plan y Amplitude

Objetivos y prerequisites

Entenderás cómo se convierte una acción de producto en un evento analizable y qué debe documentarse antes de medir funnels o retención.

Un evento es un registro de una acción: `reserva_creada`, con momento, usuario y propiedades como canal o importe. Un **tracking plan** define nombre, cuándo se envía, quién lo emite, propiedades, identidad, validaciones y propietario. Amplitude puede explorar eventos, funnels, cohortes y retención, pero no corrige un evento mal definido.

Verifica cobertura y cambios de versión antes de interpretar una caída. Si una app deja de enviar una propiedad, un dashboard puede mostrar un comportamiento inexistente.

Resumen

Medir producto es diseñar un sistema de evidencia, no pulsar “crear gráfico”.

BI, dashboards y definición de métricas

Objetivos y prerequisites

Aplicarás principios comunes a Power BI, Tableau, Looker u hojas de cálculo.

Las interfaces cambian, pero un dashboard fiable necesita modelo de datos, métrica con contrato, filtros visibles, actualización conocida, propietario y audiencia. No combines tablas con grano distinto solo para que un panel “tenga más información”.

Cada visual debe responder una pregunta y ofrecer acceso al detalle necesario para auditoría. Documenta zona horaria, exclusiones, último refresco y alertas. Un dashboard es un producto: requiere mantenimiento cuando cambian fuentes o definiciones.

Resumen

BI comunica decisiones repetidas; no reemplaza investigación ni documentación.

Entrega, seguimiento y comunicación

Objetivos y prerequisites

Cerrarás un análisis dejando claro qué se recomienda, con qué evidencia y qué ocurrirá después.

Una entrega mínima contiene decisión, hallazgo, evidencia, recomendación, límites, enlace a fuentes y siguiente medición. Adapta forma: nota ejecutiva para dirección, ticket con criterios para ingeniería, dashboard para seguimiento. No cambies el nivel de certeza por cambiar el formato.

El seguimiento convierte análisis en aprendizaje: registra la acción acordada, dueño, fecha y métrica que evaluará resultado. Si el efecto no aparece, vuelve a hipótesis, instrumentación y supuestos en lugar de defender la conclusión inicial.

Completa el [ticket analítico](#). Después podrás abordar técnicas avanzadas sin abandonar la trazabilidad profesional.

Bloque 14 - Nivel avanzado: causalidad, escala y criterio

Propósito

Reconocer problemas avanzados que aparecen al analizar productos y operaciones reales: causalidad, anomalías, datos grandes y datos externos.

Lecciones

1. Preguntas causales y diseños posibles
2. Bootstrap y sensibilidad
3. Anomalías, monitorización y alertas
4. Escala, formatos y motores analíticos
5. APIs, geoespacial y datos externos

Práctica

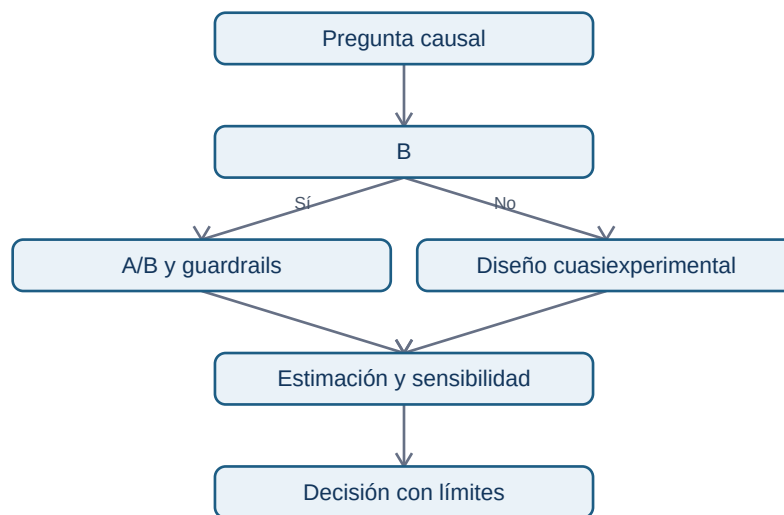
Evalúa un supuesto causal.

Preguntas causales y diseños posibles

Objetivos y prerequisites

Distinguirás “qué ocurrió” de “qué habría ocurrido si cambiamos algo” y elegirás evidencia proporcional a la decisión.

Una pregunta causal compara escenarios que no se observan a la vez: “¿reducir el formulario aumentaría reservas?”. La correlación entre formularios cortos y más reservas no basta; quizá los usuarios o campañas eran distintos. Un experimento aleatorizado aproxima una comparación justa al asignar variantes de forma controlada.



Cuando no hay experimento, diferencias en diferencias, regresión discontinua o matching pueden aportar evidencia, pero cada uno necesita supuestos verificables y análisis de sensibilidad. No son “botones de causalidad”.

Resumen

Declara el cambio, la población, el contrafactual y los supuestos. Continúa con [bootstrap y sensibilidad](#).

Bootstrap y sensibilidad

Objetivos y prerequisites

Usarás remuestreo para entender estabilidad de una estimación y variarás supuestos para comprobar si una conclusión depende de una decisión frágil.

El **bootstrap** crea muchas muestras al volver a seleccionar, con reemplazo, observaciones de los datos disponibles. Al recalcular una métrica se obtiene una distribución de estimaciones. Es útil cuando la fórmula analítica es complicada, pero no corrige una muestra sesgada ni convierte datos insuficientes en evidencia sólida.

Un análisis de sensibilidad cambia decisiones razonables: ventana temporal, tratamiento de outliers, umbral o definición de métrica. Si una recomendación se invierte con cambios pequeños, comunícalo y evita una afirmación tajante.

Resumen

Incertidumbre no es solo un intervalo: también es dependencia de datos y supuestos.

Anomalías, monitorización y alertas

Objetivos y prerequisites

Construirás una respuesta ordenada ante un valor inesperado sin confundir señal con incidente.

Una **anomalía** es una observación que se aparta de un patrón esperado. Antes de alertar a negocio, comprueba tracking, frescura de datos, calendario, despliegues y cambios de definición. Una caída de eventos puede ser un fallo de instrumentación; una subida de ventas, una campaña legítima.

Una alerta necesita métrica, referencia, umbral, ventana, responsable y runbook: qué comprobar y cuándo escalar. Umbrales demasiado sensibles producen fatiga de alertas; demasiado laxos detectan tarde. Ajusta con historial y coste de no detectar.

Resumen

Monitorizar es diseñar una decisión operativa, no solo pintar una línea roja.

Escala, formatos y motores analíticos

Objetivos y prerequisites

Elegirás una estrategia cuando los datos superen la memoria o el tiempo de una herramienta local.

Primero reduce el problema: selecciona columnas, filtra antes de transferir, agrega cerca de la fuente y evita duplicaciones. Los formatos columnares como Parquet permiten leer solo campos necesarios. DuckDB consulta archivos y tablas localmente; Polars procesa datos de manera eficiente. Son herramientas, no excusas para ignorar grano, calidad o coste.

Cuando el equipo usa warehouse, lakehouse o procesamiento distribuido, mueve cómputo cerca de los datos y controla permisos, gasto y particiones. Una consulta rápida pero semánticamente errónea sigue siendo errónea.

Resumen

Escalar empieza por una pregunta más precisa y un modelo de datos correcto.

APIs, geoespacial y datos externos

Objetivos y prerequisites

Integrarás datos externos sin perder trazabilidad, licencias ni contexto geográfico.

Una API permite que programas soliciten datos a otro servicio. Antes de usarla, registra proveedor, permiso, licencia, fecha, versión, límites, campos y transformaciones. Una fuente externa puede cambiar sin avisar o medir una población distinta a la tuya.

Los datos geoespaciales añaden ubicación, pero una coordenada no siempre indica residencia, tienda o zona de entrega. Agregar por áreas puede ocultar diferencias internas o crear riesgos de privacidad. Minimiza precisión cuando no sea necesaria y evita inferencias sensibles sobre personas.

Cierre

Evalúa el [supuesto causal](#). El último bloque convertirá el recorrido en pruebas de competencia y portfolio defendible.

Bloque 15 - Portfolio y preparación profesional

Propósito

Convertir competencias en evidencia visible: proyectos que muestran criterio, técnica, comunicación y honestidad sobre los límites.

Lecciones

1. Seleccionar y delimitar casos
2. Estructurar un proyecto defendible
3. Narrativa, revisión y publicación
4. Entrevistas, CV y capstone

Capstone

El [proyecto final](#) integra el curso: datos sin limpiar, análisis, métricas, visualización, recomendación y una entrega para público no técnico.

Cierre

Terminar el curso no significa saberlo todo. Significa tener un método fiable para aprender una herramienta nueva, hacer preguntas mejores y justificar decisiones con evidencia.

Seleccionar y delimitar casos

Objetivos y prerequisites

Elegirás proyectos que demuestren competencias reales y limitarás su alcance para poder terminarlos con rigor.

Un portfolio no es una colección de gráficos ni una lista de tecnologías. Un caso valioso empieza por una decisión: priorizar una mejora de producto, prever demanda, entender abandono o detectar una ineficiencia. Debe permitir mostrar pregunta, datos, calidad, método, evidencia, límites y recomendación.

Elige tres casos variados en vez de diez exploraciones a medias: uno de datos tabulares y limpieza; uno de métricas/producto o SQL; y uno que incorpore incertidumbre, previsión o modelo cuando sea apropiado. Usa datos con licencia y sin información personal innecesaria.

Límite

No inventes impacto empresarial ni presentes datos simulados como si fueran clientes reales. Un proyecto didáctico puede ser excelente si declara qué es simulado, qué decisión ilustraría y qué evidencia faltaría en producción.

Resumen

La calidad del problema y la honestidad del alcance pesan más que la complejidad aparente. Continúa con [estructura defendible](#).

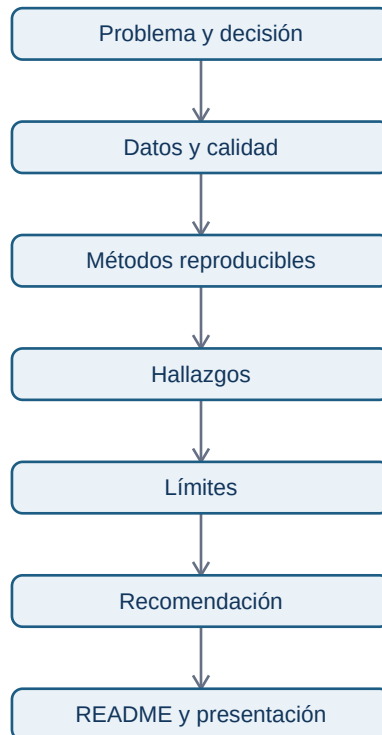
Estructurar un proyecto defendible

Objetivos y prerequisites

Organizarás los artefactos que permiten a otra persona entender, ejecutar y cuestionar un análisis.

Un proyecto debe incluir README ejecutivo, pregunta y decisión, diccionario de datos, procedencia/licencia, código o notebook reproducible, visualizaciones, límites y próximos pasos. El README no repite cada detalle técnico: permite entender en dos minutos qué se halló y dónde está la evidencia.

Este flujo responde a “¿qué debe poder seguir una persona que revisa un caso?”



El diagrama no obliga a una secuencia rígida: al hallar un error de datos puedes volver a la pregunta. Sí obliga a no saltar de datos a recomendación sin mostrar el razonamiento.

Resumen

Un caso defendible conserva tanto el resultado como el camino. Continúa con [narrativa y revisión](#).

Narrativa, revisión y publicación

Objetivos y prerequisites

Comunicarás un hallazgo a una audiencia concreta y revisarás el proyecto antes de hacerlo público.

Una narrativa analítica ordena contexto, pregunta, evidencia, interpretación, recomendación y límite. No empieza por la herramienta: “usé Python” no es un hallazgo. Un título útil dice qué ocurrió y para quién; una visualización enseña la comparación que lo respalda.

Antes de publicar, revisa enlaces, rutas, instrucciones de ejecución, dependencias, datos sensibles, licencias, definiciones de métricas y conclusiones excesivas. Pide a otra persona que intente seguir el README: si no sabe cómo reproducir o entender una decisión, el proyecto todavía no está listo.

Error habitual

Eliminar los pasos incómodos para que el portfolio parezca perfecto. Documentar una limitación o un dato descartado con razón aumenta credibilidad profesional.

Resumen

Publicar es una entrega para lectores reales, no el final de una carpeta local.

Entrevistas, CV y capstone

Objetivos y prerequisites

Prepararás una explicación oral de tu método y cerrarás el curso con un proyecto integrador.

En entrevista, empieza por aclarar decisión, población, métrica y datos disponibles. Al explicar un caso, distingue hecho observado, interpretación, riesgo y siguiente comprobación. Practica responder: “¿qué grano tiene esta tabla?”, “¿cómo validarías este JOIN?”, “¿qué harías ante un dato ausente?” o “¿por qué no demuestra causalidad este gráfico?”.

El CV y GitHub deben enlazar proyectos que puedas defender línea a línea: problema, contribución, herramientas, resultado y límite. No declares competencias que no puedas aplicar o explicar.

Capstone

El [proyecto final](#) integra el curso: datos sin limpiar, análisis, métricas, visualización, recomendación y una entrega para público no técnico. Empieza leyendo su [rúbrica](#); úsala como lista de control, no solo como nota final.

Cierre

El objetivo no es saberlo todo: es tener un método fiable para aprender una herramienta nueva, hacer preguntas mejores y justificar decisiones con evidencia.